



Universidad Técnica Estatal de Quevedo

Aplicaciones Distribuidas – Séptimo A

TiendaTech

Documento acumulativo E1–E4

Arquitectura, implementación y evaluación experimental

Proyecto Fin de Curso – Paso 13

Integrante	Rol asignado
Jhinson Stalyn Aucatoma Celorio	Arquitecto
Jeremy Ruperto Gaibor Rodriguez	Desarrollador
Andy Paul Sanchez Pilaloe	Responsable de Calidad
José Alejandro Lozano Morales	Documentalista

Docente: Gleiston Cicerón Guerrero Ulloa

Período académico 2026

Corte documental: 1 de septiembre de 2026

Índice

Resumen	v
Abstract	vi
1. Introducción	1
1.1. Objetivos y alcance	1
1.2. Corte de evidencia y organización	1
1.3. Registro de cambios explícito frente a E1	2
1.4. Registro de cambios por entrega	5
2. Estado del arte	9
2.1. Consistencia, orden y datos distribuidos	9
2.2. Coordinación transaccional: 2PC frente a Saga	9
2.3. Procesamiento paralelo y verificación de consultas	10
2.4. Microservicios y clientes	10
2.5. Pruebas y entrega continua	11
2.6. Criterio bibliográfico	11
3. Diseño del sistema	11
3.1. Contexto y contenedores	11
3.2. Fronteras de dominio	12
3.3. Componentes y capas	13
3.4. Despliegue y decisiones	14
4. Propiedades distribuidas	14
4.1. Consistencia por agregado	14
4.2. Fragmentación y réplica	15
4.3. Modelo de fallos y orden	16
5. Implementación	16
5.1. Comunicación y contratos	16
5.2. Acceso a datos y control de transacciones	16
5.3. Procesamiento distribuido	17
5.4. Interfaces y capas	17
5.5. Instrumentación	19
5.6. Trazabilidad de los temas de la asignatura	19
6. Diseño del estudio	22
6.1. Preguntas de investigación	22
6.2. Factores, unidad experimental y procedimiento	22
6.3. Variables e invariantes	23
6.4. Análisis estadístico	23
6.5. Datos y paralelismo	24
7. Resultados	24
7.1. Coordinación local	24
7.2. Procesamiento analítico	26
7.3. Persistencia y tolerancia a fallos	27

7.4. Actualización de evidencia del 3 de septiembre de 2026	28
8. Discusión	29
8.1. Qué permite concluir la comparación	29
8.2. Experimento real de checkout bajo carga (campana correctiva, 2026-09-05)	30
8.3. Paralelismo y motor de datos	31
8.4. Decisiones de cierre	32
9. Amenazas a la validez	32
9.1. Validez interna	32
9.2. Validez externa	33
9.3. Validez de constructo	33
9.4. Validez de conclusión	34
10. Calidad del producto	34
10.1. Modelo y criterios	34
10.2. Cobertura y complejidad	35
10.3. CI, carga y observabilidad	38
11. Gestión de la revisión cruzada	40
11.1. Criterio de respuesta	40
11.2. Justificación y coste de la deuda	44
12. Conclusiones	46
12.1. Respuestas a las preguntas de investigación	46
12.2. Balance acumulativo	46
13. Conclusiones individuales	47
13.1. Jhinson Stalyn Aucatoma Celorio	47
13.2. Jeremy Ruperto Gaibor Rodriguez	48
13.3. Andy Paul Sanchez Pilaloa	48
13.4. José Alejandro Lozano Morales	49
14. Reflexión ética	49
15. Declaraciones	50
15.1. Autoría y contribución	50
15.2. Uso de inteligencia artificial generativa	51
15.3. Reproducibilidad documental	51
16. Bibliografía	53

Índice de figuras

1.	C4 nivel 1: contexto del sistema implementado. Elaboración propia.	12
2.	C4 nivel 2: contenedores lógicos de aplicación. La agrupación Java resume seis servicios independientes.	12
3.	C4 nivel 3: componentes del recorrido de reserva; vista propia de responsabilidades.	13
4.	Vista de despliegue por responsabilidades. Réplica entre nodos; ensayo de clúster en un mismo equipo.	14
5.	Rutas públicas de la aplicación web TiendaTech. Evidencia propia del equipo.	18
6.	Gestión administrativa de resumen, usuarios y productos. Evidencia propia del equipo.	18
7.	Gestión administrativa de proveedores, órdenes y facturación. Evidencia propia del equipo.	18
8.	Autenticación y cámara en la aplicación Android. Evidencia propia del equipo.	18
9.	Consulta del catálogo desde la aplicación Android. Evidencia propia del equipo.	19
10.	Distribución de p95 de compra por ejecución: cinco repeticiones en cada caja; bigotes mínimo–máximo. Elaboración propia desde el CSV crudo. . .	25
11.	T1: media e intervalo del 95 % publicados, por modalidad y ejecutores. Elaboración propia.	27
12.	Tasa de confirmación y mediana de p95 entre las quince corridas de cada estrategia y concurrencia. Fuente: CSV correctivo derivado sin alterar los datos crudos.	31
13.	Dashboard Grafana con datos reales durante la sesión conjunta de carga. Fuente: evidencia propia del Paso 10.	39
14.	Trazas distribuidas exportadas durante la sesión conjunta; incluyen el recorrido instrumentado y el canal TCP. Fuente: evidencia propia del Paso 10.	39

Índice de tablas

1.	Registro de cambios frente a E1, ítems 2.1–2.8.	2
2.	Registro explícito de recuperación y correcciones E1–E4.	6
3.	Fronteras y restricciones del sistema.	13
4.	Consistencia, réplica y límites por agregado, sostenidos con el oráculo del Paso 8.	15
5.	Trazabilidad unificada de temas, fuentes y límites de evidencia.	20
6.	Protocolo solicitado y configuración realmente ejecutada.	23
7.	Latencia de compra local y confirmaciones por segundo. IC del 95 % del p95 mediano.	24
8.	Recursos y abortos de condiciones sin fallo, medianas de recursos.	26
9.	Concurrencia 400: medianas entre ejecuciones de p50, p99 y tiempo hasta compensación, en ms.	26
10.	Tiempo de T1 por motor y modalidad.	27
11.	Tiempos registrados en comparación de planes, normalizados a milisegundos.	28
12.	Comprobación del stock en las bases del piloto posterior.	28
13.	Campaña correctiva agregada por estrategia y concurrencia.	30
14.	Oráculo retrospectivo sobre la instantánea de CockroachDB.	31
15.	Evaluación por características ISO/IEC 25010:2023.	35
16.	Cobertura en el alcance de los informes disponibles.	36
17.	Respuesta y evidencia individual de revisión cruzada.	41
18.	Deuda técnica: resolución, coste de arrastre y evidencia necesaria para cerrar.	44
19.	Contribuciones por rol, sujetas a revisión final de sus autores.	50

Resumen

Objetivo: consolidar la evolución de TiendaTech y evaluar qué propiedades de su arquitectura distribuida quedan respaldadas por evidencia. **Método:** revisión del repositorio, mediciones de CockroachDB y Spark, y una campaña correctiva de 120 corridas mediante API Gateway, microservicios y CockroachDB, con dos estrategias, cuatro concurrencias y tres condiciones de fallo. **Resultados:** la matriz real contiene 208003 solicitudes y 49786 intentos de checkout, de los cuales 30275 fueron confirmados. Saga confirmó 63,4 % y 2PC 58,1 % en el agregado descriptivo; ninguna de las doce comparaciones de p95 resultó significativa con Bonferroni. Un oráculo retrospectivo sobre CockroachDB encontró 13210 órdenes con movimiento de inventario ausente o distinto entre 43168 órdenes persistidas; no halló importes discordantes, descuentos duplicados ni stock negativo, mientras la compensación quedó no verificable. La evaluación ISO observó 3588/3588 sondeos exitosos durante una hora y un p95 de 610ms con 50 usuarios, superior al objetivo de 500 ms. **Contribución:** una memoria trazable que separa implementación, medición y límites de inferencia. No se acredita una comparación reglas-RAG.

Palabras clave: sistemas distribuidos, coordinación, CockroachDB, Spark, calidad, reproducibilidad.

Abstract

Objective: to consolidate TiendaTech’s evolution and determine which distributed-system properties are supported by evidence. **Method:** repository review, CockroachDB and Spark measurements, and a corrective campaign of 120 runs through the API Gateway, microservices and CockroachDB, covering two strategies, four concurrency levels and three fault conditions. **Results:** the real matrix contains 208,003 requests and 49,786 checkout attempts, of which 30,275 were confirmed. Saga confirmed 63.4 % and 2PC 58.1 % in the descriptive aggregate; none of the twelve p95 comparisons was significant after Bonferroni correction. A retrospective CockroachDB oracle found 13,210 orders with missing or mismatched inventory movements among 43,168 persisted orders; it found no amount mismatches, duplicate discounts or negative stock, while compensation remained unverifiable. The ISO assessment observed 3,588 successful probes out of 3,588 during one hour and a p95 of 610 ms with 50 users, above the 500 ms target. **Contribution:** a traceable report separating implementation, measurement and inference limits. A rules-versus-RAG comparison is not established.

Keywords: distributed systems, coordination, CockroachDB, Spark, quality, reproducibility.

1. Introducción

TiendaTech es un proyecto académico de comercio electrónico de componentes de computación. Integra clientes web y móvil, autenticación, catálogo, inventario, pedidos, ventas, compras a proveedores y un asistente de armado. El problema técnico consiste en conservar reglas de negocio cuando la información cruza procesos y se presentan reintentos, fallos de comunicación o indisponibilidad de datos. Una compra no debe confirmar un pago distinto del esperado, descontar inventario indebidamente ni permanecer en un estado parcial sin una política de recuperación.

La cuantificación disponible corresponde al entorno experimental, no a pérdidas comerciales de una empresa. El conjunto analítico histórico incluye 600 000 pedidos y detalles, 10 000 usuarios y 570 000 filas tras el filtrado y unión. El ensayo de coordinación comprende 24 condiciones y cinco repeticiones por condición, con concurrencias nominales entre 50 y 400. Estos tamaños delimitan la evaluación: no se dispone de una línea base de incidentes ni de clientes reales que permita estimar impacto económico. La ausencia de esos datos no se reemplaza por cifras supuestas.

1.1. Objetivos y alcance

El objetivo general es documentar y evaluar una implementación distribuida reproducible, identificando sus garantías y límites. Los objetivos específicos son: describir las fronteras y decisiones arquitectónicas; relacionar consistencia, replicación y fallos con agregados del negocio; presentar mediciones de procesamiento y coordinación; contrastar calidad y revisión cruzada con evidencia; y responder las preguntas del banco experimental sin extrapolar más allá de su implementación.

La memoria acumula E1 (propuesta y decisiones), E2 (modelo y distribución de datos), E3 (integración y análisis) y E4 (refactor, seguridad, pruebas y evaluación). El nombre anterior del repositorio fue **PFC-AppsDistribuidas**; la denominación adoptada es **TiendaTech**. Esta aclaración conserva la interpretación de rutas y mensajes históricos. Las tres capturas de búsqueda del nombre se conservan en **docs/nombre/** desde el commit **ea0b4eb** del 26 de agosto. El buscador general y GitHub muestran coincidencias; SourceForge muestra cero resultados con el filtro Windows activo. Se documenta la búsqueda realizada, sin inferir exclusividad de la denominación. Su inventario y alcance están enlazados desde el README.

1.2. Corte de evidencia y organización

El corte histórico de la macro `\evidencia` integra las evidencias publicadas hasta el commit **c5dca58**, cuyo hash completo fijan sus enlaces. Los reportes web posteriores se citan mediante enlaces individuales a commits publicados con hash completo, según el registro de verificación del 12 de septiembre. Incluye la campaña correctiva del 5 de septiembre, la evaluación ISO, las pruebas de CI, el panel Grafana y las trazas. Cada resultado conserva su procedencia y unidad experimental; el cierre de esta memoria no equivale a declarar aprobados los pasos técnicos que permanecen como deuda.

El texto avanza desde el estado del arte y el diseño hasta la implementación; define después

el estudio, presenta resultados separados de su discusión y examina amenazas, calidad y revisión cruzada. Las conclusiones responden las preguntas experimentales; las reflexiones individuales, ética y declaraciones explicitan la responsabilidad del equipo. La bibliografía sigue estilo IEEE con Biber. Los identificadores de evidencias permiten localizar el dato original y diferenciarlo de su interpretación.

1.3. Registro de cambios explícito frente a E1

La tabla 1 responde al ítem 2.1 del Paso 2: qué cambió respecto de la propuesta de E1 (*TiendaTech: Propuesta y Análisis del Sistema Distribuido*, 27 de mayo de 2026) y por qué. No se trata de una lista de novedades técnicas sueltas, sino de la evolución de cada uno de los ocho compromisos de E1 hasta el corte documental integrado. Dos ítems se marcan explícitamente como parciales o pendientes: no se declara cumplido lo que todavía no tiene evidencia cerrada.

Tabla 1: Registro de cambios frente a E1, ítems 2.1–2.8.

Ítem	En E1	Qué cambió en E4 y por qué
2.1 Problema cuantificado	Cuantificación de mercado: facturación de e-commerce nacional (CECE, >4 000 M USD), hardware entre los tres sectores de mayor volumen. Motivación de negocio, no medida por el equipo.	La Introducción sustituye la cifra de mercado por la cuantificación del propio entorno experimental: 600 000 pedidos y detalles, 10 000 usuarios, 570 000 filas tras filtrado, 24 condiciones $\times$ 5 repeticiones del ensayo de coordinación. Se abandona una cita externa no verificable por el equipo y se sostiene el problema con datos que el propio proyecto puede respaldar: “la ausencia de esos datos no se reemplaza por cifras supuestas”.
2.2 Justificación de arquitectura	ADR-01 (E1): microservicios justificados por separación de responsabilidades y “reducir el impacto de fallos”, consecuencias enunciadas como expectativa.	“Despliegue y decisiones” agrega la distinción explícita entre decisión y medición: “no todas las consecuencias previstas en E1 se convirtieron en resultados medidos [...] demostrar que mejora latencia o disponibilidad requiere un experimento”. Cambio de una justificación cualitativa a una que declara qué de lo prometido en E1 sigue sin comprobarse.

Ítem	En E1	Qué cambió en E4 y por qué
2.3 Estado del arte ≥ 15 fuentes	Matriz de 7 referencias (Cuadro 1 de E1), organizada en torno al problema de negocio (recomendación de catálogo, colisiones de compatibilidad, NLP/LLM).	Cuatro subsecciones nuevas (consistencia y datos distribuidos, procesamiento paralelo, microservicios y clientes, pruebas y CI/CD) que inicialmente citaban 15 fuentes con DOI; la ampliación documental añade cinco específicas de 2PC y Saga. El estado del arte se reorienta de la motivación de negocio de E1 a la literatura que sostiene las decisiones técnicas realmente evaluadas en E2–E4 (CAP, relojes lógicos, Amdahl/Gustafson, CI/CD).
2.4 Diagramas C4	Figuras 1 (C4-L1) y 2 (C4-L2) insertadas como imagen, sin nivel de componentes (L3).	“Contexto y contenedores” y “Componentes y capas” reconstruyen L1 y L2 y agregan L3, con TikZ embebido y versionado como código fuente en vez de imagen suelta. La reconstrucción es necesaria porque la arquitectura real cambió (seis servicios Java, armado-ia en Python, gateway), no una actualización estética.
2.5 ADR	Tres ADR fundacionales: 01 (microservicios), 02 (API Gateway), 03 (motor de compatibilidad).	El corte referencia docs/adr con ADR-003 a ADR-012 (fragmentación temporal de pedidos, Raft, patrones de diseño, JWT, capas Python, recuperación de las tres decisiones de E1, fragmentación del catálogo) más dos registros móviles. De 3 decisiones fundacionales a un registro vivo de 12+ decisiones que cubre persistencia distribuida, seguridad y despliegue.

Ítem	En E1	Qué cambió en E4 y por qué
2.6 Posición CAP por agregado (datos del Paso 8)	No existe tabla CAP; el ADR-01 solo menciona “alta disponibilidad” en abstracto, sin diferenciar por agregado.	La tabla 4 (“Consistencia por agregado”) formula una política CAP distinta para inventario, pedido/pago, usuarios, catálogo y analítica, cada una con su límite de evidencia declarado. Actualización del 7 de septiembre: la campaña correctiva de checkout produjo 30275 confirmaciones en 120 corridas contra el sistema desplegado, después de aprobar pilotos basales y una rampa de carga, y un oráculo retrospectivo auditó esas mismas corridas directamente contra CockroachDB real (no contra el CSV agregado). El oráculo evalúa parcialmente invariantes por agregado, con límite declarado, pero no valida CAP porque no se indujo una partición de red controlada: cero discordancias de importe entre orden y factura, cero descuentos duplicados y cero stock negativo, frente a 30,6 % de órdenes con inconsistencia de inventario. Ninguna de las dos estrategias (2PC, Saga) garantizó por sí sola esa consistencia de inventario bajo esta concurrencia, y la comparación entre ambas no fue estadísticamente significativa tras Bonferroni. La cuarta invariante (compensación de intentos cancelados) sigue sin poder verificarse, no por falta de intento sino porque su registro vivía solo en memoria y se perdió entre reinicios; se declara <code>no_verificado</code> en vez de asumirse cumplida.

Ítem	En E1	Qué cambió en E4 y por qué
2.7 Modelo de fallos de Cristian	No existe modelo de fallos ni de sincronización de reloj en E1.	“Modelo de fallos y orden” cubre omisión, demora, caída de proceso y reintentos, y formaliza el <i>orden</i> causal con relojes lógicos de Lamport (ecuación 1). Brecha real, no resuelta: el ítem pide el algoritmo de Cristian (sincronización de reloj físico mediante intercambio de mensajes con un servidor de tiempo); el documento no lo menciona ni lo implementa. Lamport resuelve un problema relacionado pero distinto (orden lógico, no sincronización de reloj físico); no debe declararse cumplido este punto sin trabajo adicional.
2.8 Métricas y plan de validación	Cuadro 2 de E1: 6 métricas ISO/IEC 25010 con umbral objetivo, sin plan de medición (sin repeticiones, descartes ni tratamiento estadístico).	“Diseño del estudio” agrega 5 preguntas operativas, la tabla 6 que compara explícitamente lo pedido por la guía frente a lo realmente ejecutado (declarando discrepancias, p. ej. retardo de 0,05 s frente a 5 s pedidos), variables con oráculo de invariantes y análisis estadístico con bootstrap, Wilson, Mann–Whitney y tamaño de efecto $\hat{A}_{12}$. De un cuadro de métricas objetivo sin plan, a un protocolo ejecutado y confrontado honestamente contra lo prometido.

1.4. Registro de cambios por entrega

La tabla 2 hace explícita la recuperación acumulativa solicitada en C1. Las bases históricas son docs/entrega1/entrega1.pdf, docs/entrega2/entrega2.pdf y docs/entrega3/PFC3.tex. La entrega de origen identifica el contenido recuperado; no significa que todos sus cambios se hayan realizado durante esa entrega. Los commits enlazados registran correcciones posteriores integradas en E4. Las reservas describen lo que todavía no queda demostrado. Las filas con prefijo «Eval.» corresponden a observaciones de evaluación, no a las entregas E1–E4; registran las correcciones del 11 de septiembre de 2026.

Tabla 2: Registro explícito de recuperación y correcciones E1–E4.

Origen	Base o aspecto revisado	Cambio integrado y límite	Evidencia histórica
E1	Propuesta de comercio electrónico y asistente; alcance inicialmente prospectivo.	Se acota la evaluación a datos sintéticos y propiedades medidas. Las promesas de disponibilidad, pagos y asistente no se presentan como resultados experimentales. Integrado en Introducción, Diseño y Conclusiones.	1b9a8e3 : recuperación E1; c766872 : síntesis acumulativa.
E1	Justificación de microservicios, gateway y compatibilidad.	Se formalizan contexto, alternativas y consecuencias en ADR-009, 010 y 011; se corrige el cableado OTP del diagrama. La justificación no acredita rendimiento ni precisión del asistente.	1b9a8e3 ; sección Diseño del sistema.
E2	Diseño C4, REST/OpenAPI y contenerización.	Se actualizan las vistas al producto integrado y los contratos por servicio. Quedan pendientes las rutas heredadas de productos y contratos completos con DTO.	47be7cc ; docs/api/; issues 64, 76 y 77.
E2	Separación de servicios y control de acceso.	Se introducen capas y puertos, se centraliza JWT en gateway y se documenta su límite de confianza. La propiedad de datos se explicita mediante migraciones; no se confunde aislamiento lógico con aislamiento físico.	17679ab ; 7dcbb11 ; ADR-007/008 y migraciones V004–V006.
E3	Distribución de datos, fragmentación y clúster.	Se recuperan las decisiones de rangos temporales y Raft, se formaliza la fragmentación del catálogo y se conserva el ensayo de caída/reintegración. Un clúster en un anfitrión no demuestra disponibilidad prolongada.	384e354 ; ADR-003/004/012; resultados de persistencia.
E3	Pipeline analítico y comparación de rendimiento.	Se actualizan transformaciones Spark, lectura JDBC, datos crudos y análisis. Se distingue lectura particionada y sin partición y se conserva la desaceleración observada, sin convertir Amdahl en una predicción confirmada.	660598f ; spark/pipeline.py; resultados/tiempo_crudos.csv.

Origen	Base o aspecto revisado	Cambio integrado y límite	Evidencia histórica
E4	Banco de coordinación y análisis central.	Se incorpora una campaña correctiva de 120 corridas distribuidas con preflight, rampa, cinco minutos de medición y 30275 confirmaciones. Se publican tablas, gráfica, prueba exacta con empates y Bonferroni. El oráculo persistente de invariantes ya se ejecutó de forma retrospectiva contra CockroachDB real: halló 30,6 % de órdenes con inconsistencia de inventario y cero discordancias de importe, descuentos duplicados o stock negativo; la compensación de intentos cancelados queda no_verificado por pérdida del registro en memoria entre reinicios.	62f0b15 ; 163f0cb ; sección 8.2; tabla 4; Amenazas y Conclusiones actuales.
E4	Integración documental, revisión y reproducción.	Se consolida la memoria, se incorporan declaraciones, DOI y comandos de reproducción. Esta revisión añade registro de cambios, trazabilidad por líneas y costes de arrastre; no cierra incidencias ni acredita nuevas pruebas técnicas.	c766872 ; 96a350b ; tablas 5 y 18.
Eval. E8	Carpetas huérfanas de la raíz.	Se retiran los archivos versionados de usuarios/ y frontend/ , y se documenta la estructura activa. La limpieza del árbol no constituye una nueva prueba funcional.	e4027c0 ; README raíz.
Eval. E4	Registro de imágenes.	Se migra de Docker Hub a GHCR, con token del workflow y etiquetas por commit, sin latest . Se conservan capturas del flujo, ejecución satisfactoria y paquete público; no se garantiza el éxito de publicaciones futuras.	a922bf4 ; 3c74323 .
Eval. E6	Paquete Android instalable.	Se incorpora un APK release firmado por José, con checksum, certificado público e instrucciones en release/ . CI exige los controles de calidad, compila con secretos protegidos, comprueba la huella del certificado y publica APK, suma, firma y procedencia. José confirmó instalación e inicio en dos dispositivos; no equivale a una prueba integral de compra.	f71bec4 ; c3edc7a ; CI ; release/README.md .

Origen	Base o aspecto revisado	Cambio integrado y límite	Evidencia histórica
Eval. E2	Citas de seguridad y reportes web.	Cita de seguridad corregida: ocho rutas con HTTP 401 en el CSV real del 11 de septiembre; once pruebas en la suite, según reporte local no versionado. Se corrigen por separado los tres enlaces web con commits publicados. No se acredita el validador automático ni el cierre global de E2.	b8d3e23 ; cierre/verificacion-referencias-20260912.md ; tabla 18.
Eval. E1/E3	Cobertura y análisis estático web.	Se publican cobertura de 19 pruebas y reporte ESLint integrado en CI. Las cifras corresponden al alcance instrumentado con HTTP simulado; no acreditan cobertura integral ni sustituyen las mediciones backend.	3e07382 ; a9d4fbb .
Eval. E1/E3	Ampliación de cobertura web al árbol completo.	Se retira la lista de doce vistas de <code>coverage.include</code> y se mide todo <code>src/**/*.{ts,tsx}</code> : 62 pruebas (antes 19), líneas 71,96 %, sentencias 56,26 %, ramas 45,10 % y funciones 44,47 %, superando la fila anterior. Se documenta que el 71,96 % de líneas está inflado por vistas escritas en pocas líneas físicas con mucho JSX/lógica condensados; sentencias y ramas son las cifras más representativas. <code>App.tsx</code> y <code>src/services/cart.ts</code> quedan en 0 %, visibles por primera vez con el alcance ampliado.	ef374a0 : ampliación de alcance y regeneración; a6dfe80 : nota sobre la inflación de la métrica de líneas.

Los cambios documentales del 2 de septiembre pertenecen al árbol de trabajo de esta revisión y aún no tienen un commit propio. Su registro no atribuye esas correcciones retrospectivamente al commit evaluado.

La actualización documental del 11 de septiembre se encuentra en el árbol de trabajo, sin commit propio. Los hashes anteriores acreditan las correcciones técnicas incorporadas; no se atribuye a ellos esta edición del manuscrito.

2. Estado del arte

2.1. Consistencia, orden y datos distribuidos

Özsu y Valduriez [1] proporcionan el marco de fragmentación, replicación y procesamiento distribuido. En TiendaTech se usa para distinguir un esquema lógico de una distribución física: crear esquemas por servicio no demuestra que los datos estén fragmentados entre nodos. Taft et al. [2] describen CockroachDB como SQL distribuido resiliente; su pertinencia está en conectar transacciones y réplica, no en trasladar resultados de esa publicación a un clúster académico.

Gilbert y Lynch [3] formalizan el conflicto entre consistencia y disponibilidad bajo particiones. Brewer [4] revisita su interpretación y evita tratar CAP como una elección estática de dos letras para todo el producto. Abadi [5] añade que también existen compromisos de latencia y consistencia durante el funcionamiento normal. En consecuencia, esta memoria separa la escritura de inventario, que requiere control de concurrencia, de la lectura de catálogo y de los resultados analíticos que admiten desfase.

Lamport [6] fundamenta el orden causal mediante relojes lógicos. Su aplicación al intercambio de reservas permite ordenar eventos relacionados, pero no convierte un timestamp en un mecanismo de exclusión mutua ni en consenso. Un reloj creciente tampoco evita por sí solo que el receptor aplique dos veces una operación. Esta distinción explica por qué la idempotencia de extremo a extremo permanece como deuda separada.

2.2. Coordinación transaccional: 2PC frente a Saga

La comparación requiere separar confirmación atómica, aislamiento y recuperación. Garcia-Molina y Salem [7] introducen las sagas como secuencias de transacciones que pueden intercalarse con otras y cuya ejecución parcial se remedia mediante compensaciones. Este fundamento permite distinguir la compensación de una operación ya confirmada del rollback de una transacción todavía abierta. Su objeto original son transacciones de larga duración; no es una evaluación del despliegue de TiendaTech ni una prueba de que cualquier implementación con pasos locales preserve sus invariantes.

Gray y Lamport [8] estudian el acuerdo de commit o aborto entre participantes y explican el bloqueo de 2PC ante el fallo de su coordinador. Comparan ese mecanismo con Paxos Commit mediante mensajes, demoras y escrituras persistentes. Para TiendaTech, esto implica que el número de commits locales no basta para representar el coste distribuido: deben observarse comunicación, estado preparado y recuperación. Tampoco debe confundirse el protocolo de confirmación con una garantía de aislamiento independiente del control de concurrencia de sus participantes. La implementación denominada E-2PC carece de esa separación de procesos y no reproduce el modelo completo del artículo.

Štefanko et al. [9] examinan Axon, Eventuate ES, Eventuate Tram y MicroProfile LRA sobre un escenario común, considerando soporte transaccional, complejidad y rendimiento. La evaluación muestra por qué interesa distinguir el patrón de la tecnología concreta que lo ejecuta. Una limitación de su comparación es que el escenario de mayor carga publica la mejor de tres ejecuciones, no una distribución completa. Sus tiempos no son una referencia numérica directamente comparable con el p95 de TiendaTech, ni constituyen un contraste

experimental directo entre nuestro E-2PC y E-Saga.

Dürr et al. [10] amplían la selección de tecnologías de saga con un catálogo de criterios que incluye compensación, persistencia, recuperación, monitorización y pruebas. Su estudio presupone que saga ya resulta adecuada para el caso de uso; no decide si debe preferirse a 2PC. Esta distinción evita utilizar una evaluación de herramientas como argumento para elegir un protocolo. En TiendaTech, una menor latencia debe contrastarse también con la posibilidad de recuperar una compra interrumpida y reconstruir su historial, capacidades que el banco actual no demuestra de extremo a extremo.

Daraghmi et al. [11] abordan la falta de aislamiento de lectura de saga mediante una propuesta con *quota cache* y un servicio *commit-sync*. El resumen de los autores informa una evaluación sobre comercio electrónico; aquí se utiliza únicamente para identificar el problema y la solución propuesta, sin trasladar cifras ni atribuir garantías generales a su mecanismo. La propuesta modifica el tratamiento del estado antes de persistirlo: no equivale a la saga convencional ni al candado global de nuestro laboratorio. Su pertinencia es recordar que permitir pasos locales no elimina por sí solo las anomalías entre operaciones concurrentes.

En conjunto, estas fuentes delimitan cuatro dimensiones para la comparación: latencia, invariantes observadas, recuperación y coste operativo. La inferencia para este proyecto es que ambos protocolos deben evaluarse bajo los mismos fallos y con participantes independientes, conservando sus controles propios y registrando estados intermedios y finales. No se busca provocar una inconsistencia para confirmar una expectativa: se requiere un instrumento capaz de detectarla si ocurre. La ausencia de anomalías solo es interpretable junto al alcance del oráculo y a las intercalaciones que el ensayo realmente permite.

2.3. Procesamiento paralelo y verificación de consultas

Amdahl [12] establece un límite para acelerar una carga fija cuando existe una parte serial. Gustafson [13] cambia el planteamiento al considerar cargas escaladas. No son predicciones intercambiables: los CSV del proyecto mantienen el conjunto de datos y por ello el análisis principal es de escalabilidad fuerte. Si Spark tarda más con más ejecutores, aumentar artificialmente la carga para obtener otra conclusión alteraría la pregunta inicial.

Rigger y Su [14] proponen particionar consultas para encontrar errores lógicos en motores de bases de datos. Esta partición de predicados no equivale a fragmentar físicamente tablas ni a dividir lecturas JDBC. Su aporte metodológico al proyecto es la necesidad de comprobar resultados equivalentes al comparar planes, no asumir que una consulta rápida es correcta. Los conteos y agregados deben conservarse antes de atribuir una ventaja al tiempo de ejecución.

2.4. Microservicios y clientes

Márquez et al. [15] revisan patrones y tácticas de microservicios; Zhong et al. [16] investigan diseño guiado por dominio. Ambas perspectivas orientan a justificar límites por responsabilidad y dependencia, no solo por el número de carpetas o contenedores. Para TiendaTech, pedidos y ventas tienen fronteras conceptuales distintas, aunque compartir una base física conserve acoplamientos operativos.

Nawrocki et al. [17] comparan enfoques nativos y multiplataforma para aplicaciones móviles. El proyecto utiliza un cliente Android y debe juzgar esa decisión según integración con el dispositivo, mantenibilidad y pruebas, sin atribuirle superioridad universal. Las capturas muestran interfaces, mientras que una garantía de compatibilidad requiere ejecutar sobre dispositivos y versiones concretos.

2.5. Pruebas y entrega continua

Shahin et al. [18] distinguen integración, entrega y despliegue continuos. La distinción impide presentar cualquier workflow exitoso como una publicación segura del producto. Rostami Mazrae et al. [19] estudian uso y migración de herramientas CI/CD; para el proyecto, el valor está en conservar controles y artefactos verificables, no en acumular proveedores de automatización.

Hui et al. [20] sintetizan desafíos y brechas de prueba de microservicios. Este marco explica por qué una alta cobertura unitaria del dominio no reemplaza contratos entre servicios, pruebas de recuperación o recorridos de usuario. La síntesis de estas veinte fuentes conduce a un criterio transversal: una decisión debe tener una justificación y una evidencia del mismo nivel de abstracción. Ni una prueba local demuestra disponibilidad distribuida ni una captura de pantalla demuestra corrección transaccional.

2.6. Criterio bibliográfico

Las veinte fuentes académicas anteriores disponen de DOI. Cinco referencias específicas de coordinación se incorporaron el 3 de septiembre de 2026; sus metadatos y el alcance de consulta se conservan en `cierre/bibliografia-2pc-saga.json`. Las normas ISO y el código de ética se citan adicionalmente por su identificador y sitio institucional, sin inventar un DOI. Por tanto, se cumple el mínimo de literatura académica con DOI, pero la lectura literal de que *toda* referencia debe tenerlo exige una excepción para estas fuentes normativas solicitadas por la propia guía. La verificación bibliográfica identifica el trabajo; no implica que se haya reproducido su experimento.

3. Diseño del sistema

3.1. Contexto y contenedores

Las figuras 1 y 2 son vistas propias, reconstruidas para este corte. La primera delimita el sistema respecto de sus usuarios; la segunda representa unidades desplegables y sus dependencias. Se evita representar una pasarela bancaria real como integrada: los pagos del banco experimental son simulados.

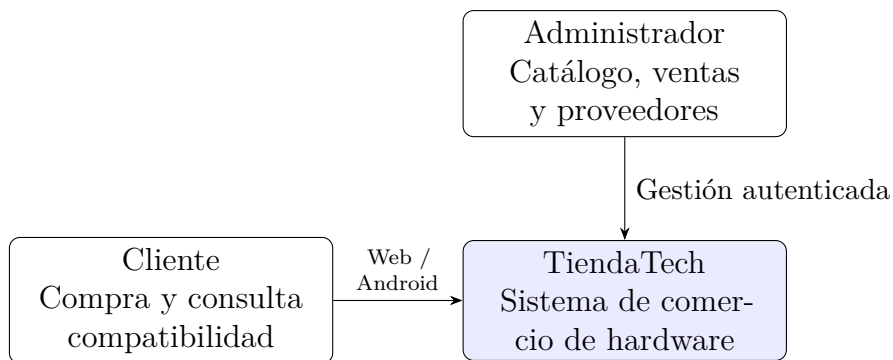


Figura 1: C4 nivel 1: contexto del sistema implementado. Elaboración propia.

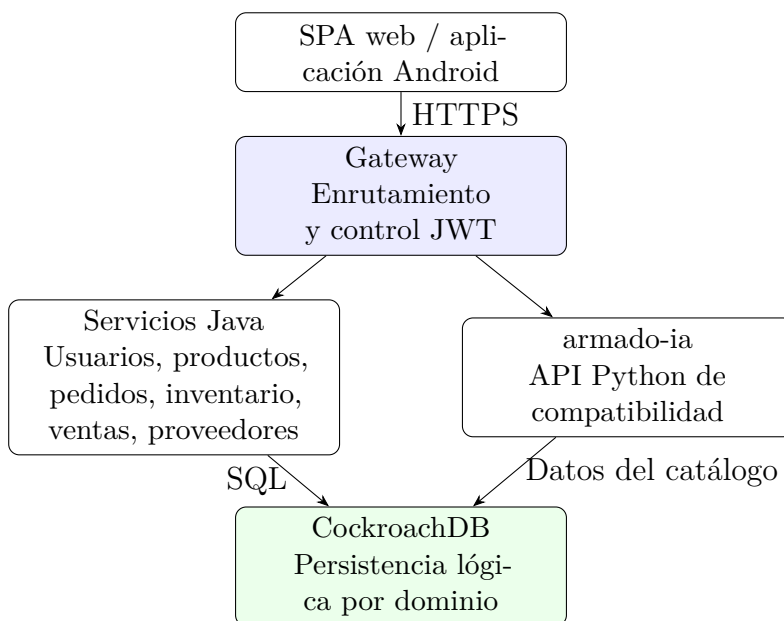


Figura 2: C4 nivel 2: contenedores lógicos de aplicación. La agrupación Java resume seis servicios independientes.

3.2. Fronteras de dominio

La tabla 3 describe la responsabilidad principal. Los nombres cortos son lógicos; no sustituyen los nombres concretos de Compose. Separar responsabilidades no garantiza aislamiento físico, puesto que las conexiones dependen del clúster compartido. Un cambio de clave en pedidos puede propagarse a consumidores aunque cada servicio tenga su propio paquete.

Tabla 3: Fronteras y restricciones del sistema.

Servicio	Responsabilidad	Restricción relevante
Usuarios	Identidad y autenticación	No publicar claves ni tokens en evidencia.
Productos	Catálogo, precios y medios	No es propietario del saldo de inventario.
Inventario	Stock y reservas	Debe impedir descuentos duplicados y negativos.
Pedidos	Carrito y órdenes	Coordina solicitudes; no equivale a transacción global.
Ventas	Registro comercial y facturación	Debe preservar importes y referencias.
Órdenes a proveedores	Abastecimiento	No confundir compra al proveedor con venta al cliente.
armado-ia	Reglas y asistencia de armado	Resultado sujeto a catálogo y reglas disponibles.
Gateway	Acceso y rutas	Es control de entrada, no regla de negocio.

3.3. Componentes y capas

La figura 3 presenta la reserva de stock como una vista de componentes; no pretende mostrar todas las clases del sistema. El contrato compartido define el mensaje mientras que aplicación y dominio conservan las reglas. El refactor Java organiza **presentation**, **application**, **domain** e **infrastructure**. En Python se documenta la correspondencia de responsabilidades con sus convenciones de módulos, sin afirmar que todos los directorios tengan los mismos nombres.

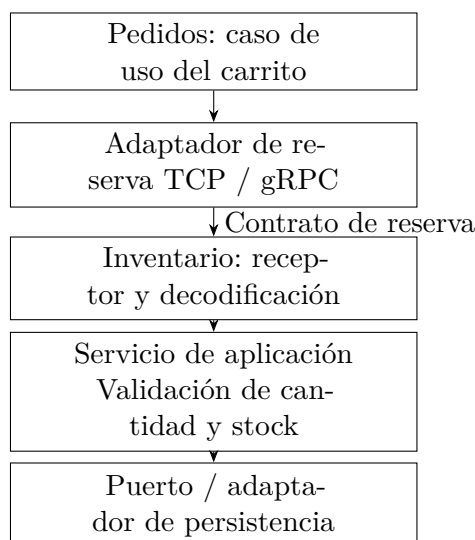


Figura 3: C4 nivel 3: componentes del recorrido de reserva; vista propia de responsabilidades.

3.4. Despliegue y decisiones

La figura 4 separa aplicación, persistencia y observabilidad. Los tres nodos del ensayo local no son tres centros de datos ni tres dominios de fallo independientes. La conexión segura del despliegue no debe confundirse con el entorno de pruebas de integración, que puede usar un único contenedor temporal.

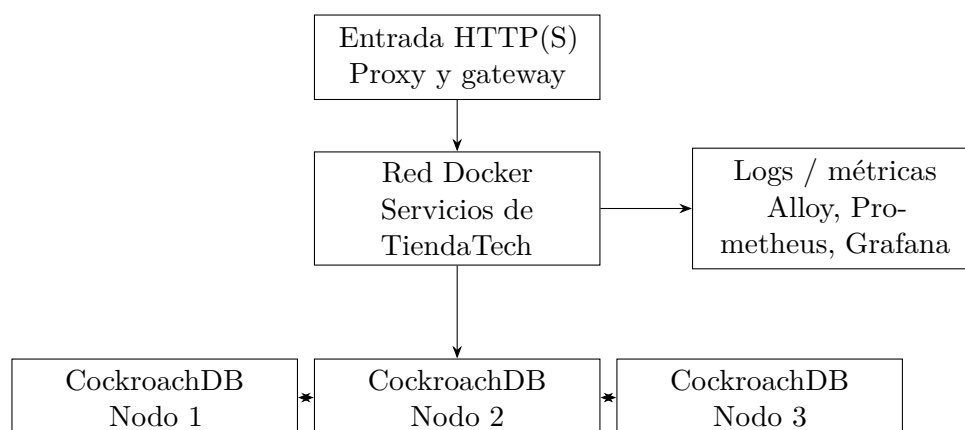


Figura 4: Vista de despliegue por responsabilidades. Réplica entre nodos; ensayo de clúster en un mismo equipo.

Los [ADR versionados](#) conservan contexto, decisión y consecuencias. ADR-003 explica la fragmentación temporal de pedidos; ADR-004 aborda Raft; ADR-005 registra patrones de diseño; ADR-007 centraliza JWT; ADR-008 describe capas Python y el límite de confianza de armado-ia. ADR-009 recupera la decisión de microservicios, ADR-010 el gateway y ADR-011 el motor de compatibilidad. ADR-012 fragmenta el índice del catálogo por categoría sin cambiar su clave primaria. Las decisiones móviles están en los registros 001 y 002.

No todas las consecuencias previstas en E1 se convirtieron en resultados medidos. Escalar un servicio por separado es una posibilidad arquitectónica; demostrar que mejora latencia o disponibilidad requiere un experimento. Asimismo, un ADR que justifica JWT en el gateway no hace seguro un servicio accesible directamente desde redes no confiables. Su validez depende de la exposición real y de pruebas negativas de acceso.

4. Propiedades distribuidas

4.1. Consistencia por agregado

La tabla 4 formula la política requerida por agregado y ahora la sostiene con datos medidos, no solo argumentados: el oráculo retrospectivo del Paso 8 auditó las 120 corridas de la campaña correctiva contra CockroachDB real ([experiments/paso8/resultados-reales/correctiva-20260905-final-v2/analisis/oracle_resumen_crdb.json](#)). CAP se interpreta ante particiones de red: la réplica por quórum puede rechazar o demorar escrituras cuando no dispone de mayoría. No se promete disponibilidad total junto con consistencia fuerte durante cualquier partición [3], [4], [5].

Tabla 4: Consistencia, réplica y límites por agregado, sostenidos con el oráculo del Paso 8.

Agregado	Política	Evidencia medida
Inventario	Serializar cambios y comprobar stock; réplica del motor.	El oráculo halló 13 210 de 43 168 órdenes persistidas (30,6 %) con movimiento de inventario ausente o distinto al esperado: la idempotencia entre servicios no se sostiene bajo esta carga. Cero descuentos duplicados y cero stock negativo en el mismo periodo.
Pedido y pago	Confirmar solo con condiciones satisfechas; compensar fallo.	Cero discordancias de importe o de líneas entre orden y factura en las 36 878 órdenes con factura persistida. La comparación 2PC/Saga no mostró diferencia significativa tras la corrección de Bonferroni ($\hat{A}_{12}$ entre 0,6 y 0,92 sin ajustar); la compensación de intentos cancelados queda no_verificado por pérdida del registro en memoria entre reinicios.
Usuarios	Preservar identidad y validar autorización.	JWT no replica estado ni revoca instantáneamente por sí solo; no evaluado por el oráculo de esta campaña.
Catálogo	Lectura consistente según consulta; réplica física.	No se midió un contrato de frescura de caché en esta campaña.
Analítica	Instantánea o extracción identificable.	Resultados derivados pueden quedar desfasados respecto de ventas; no evaluado en esta campaña.

El agregado de facturación sostiene consistencia fuerte bajo esta carga: cero violaciones de importe en el subconjunto auditable. El agregado de inventario no la sostiene: la tasa de inconsistencia observable del 30,6 % indica que la garantía declarada no se cumplió en esta campaña, y esa es precisamente la variable central que el Paso 8 exige medir. La elección entre 2PC y Saga no puede fundamentarse todavía en una diferencia de rendimiento estadísticamente significativa; sí puede fundamentarse en que ninguna de las dos estrategias, tal como están implementadas, garantizó la consistencia de inventario bajo esta concurrencia.

4.2. Fragmentación y réplica

Pedidos utiliza límites temporales de rangos; el catálogo divide el índice secundario por categoría. ADR-012 documenta un conjunto sintético de 150 productos y diez rangos, con réplicas votantes en los nodos 1, 2 y 3. Es fragmentación del índice, no una demostración de partición exclusiva de todo el agregado por nodo. El identificador del producto se mantiene para no romper referencias desde carrito, medios e inventario.

La tolerancia depende de la mayoría disponible y del tipo de fallo. Perder un nodo

de tres permite conservar mayoría en la configuración ensayada, pero una caída del equipo anfitrión puede afectar los tres simultáneamente. La replicación tampoco reemplaza copias de seguridad: una modificación lógica incorrecta puede replicarse. La evidencia de reintegración se presenta en resultados sin convertirla en un porcentaje de disponibilidad anual.

4.3. Modelo de fallos y orden

Se consideran omisión de respuesta, demora, caída de proceso y reintentos. El laboratorio del Paso 8 representa omisión y demora mediante excepciones locales; no corta paquetes en una red real. No se simularon nodos maliciosos, pérdida total del anfitrión, corrupción persistente ni recuperación de un coordinador tras reiniciar durante un commit.

El reloj lógico cumple la regla de recepción

$$L_{\text{nuevo}} = \text{máx}(L_{\text{local}}, L_{\text{recibido}}) + 1. \quad (1)$$

La ecuación 1 establece un orden compatible con causalidad [6]; no mide duración física. Para medir latencia se emplea un reloj monotónico local. La deduplicación requiere un identificador de operación persistente y una política de reintento, además del reloj.

5. Implementación

5.1. Comunicación y contratos

La reserva TCP utiliza longitud de cuatro bytes en orden de red seguida por el JSON. Tanto emisor como receptor deben completar la lectura; una sola llamada no garantiza recibir un mensaje íntegro. La variante gRPC define el contrato en [stock_reservation.proto](#). Los stubs deben regenerarse durante la compilación.

La disponibilidad de ambas rutas no demuestra que una sea más rápida. Los ficheros del experimento TCP/gRPC deben contener observaciones y no solo cabeceras antes de usar sus percentiles. El banco de coordinación es un experimento distinto y no completa esa comparación de transportes.

5.2. Acceso a datos y control de transacciones

El extracto 1 procede del laboratorio, no del servicio productivo. Corrige una versión anterior de este documento, que describía un bloqueo compartido de Python (`threading.Lock`) serializando todas las compras del proceso, incluso las que no conflictuaban entre sí. Esa serialización artificial fue eliminada del corte vigente (commit 5905639, “corregir concurrencia tiempos y repeticiones”): el control de escritura recae únicamente en SQLite, mediante `BEGIN IMMEDIATE` (que reserva el archivo de base de datos para el conector que lo emite) y `PRAGMA busy_timeout=30000` (que hace esperar a un conector en conflicto en vez de fallar de inmediato). La diferencia no es cosmética: el bloqueo de Python medía tiempos de un sistema efectivamente secuencial en todo el proceso, mientras que el bloqueo de SQLite permite que operaciones sobre productos distintos progresen en paralelo.

```

def two_phase_commit(self, tx, case, started):
    with closing(self.connect()) as db:
        db.execute("BEGIN IMMEDIATE")
        try:
            # Líneas omitidas: stock, votos e inyector.
            ...
            db.commit()
        except Exception:
            db.rollback()
            raise

```

Listing 1: Extracto de E-2PC local: control de escritura delegado en SQLite, sin bloqueo de Python.

Saga no conserva un bloqueo global durante todo su recorrido, a diferencia de lo que afirmaba la versión previa de este texto. Reserva el stock dentro de una única transacción SQLite y la confirma de inmediato; solo después intenta capturar el pago. Si el pago falla, compensa mediante una secuencia de escrituras independientes en modo autocommit, no dentro de una segunda transacción atómica: esa asimetría es intencional en el patrón Saga, donde cada paso se confirma por separado y la recuperación ocurre por acciones compensatorias explícitas, no por el rollback de una transacción larga como en 2PC.

Los puntos suspensivos identifican líneas omitidas; el código completo está en [coordination_lab.py](#). La misma corrección añadió un invariante al oráculo, `stock_cuadra_con_movimientos`, que compara el stock vigente contra el stock inicial más la suma de movimientos registrados; el oráculo anterior no ejecutaba esta comprobación de consistencia global. No existen en este recorrido tres gestores transaccionales independientes ni recuperación durable del coordinador comparable a un protocolo 2PC distribuido.

5.3. Procesamiento distribuido

El pipeline analítico calcula transformaciones equivalentes con Pandas y PySpark. La comparación conserva tarea, tamaño y resultado lógico. La lectura JDBC sin particiones limita el paralelismo de entrada; añadir ejecutores no garantiza aprovecharlos. Una variante particionada se conserva como ensayo diferente, con tres repeticiones, y no se mezcla con las cinco del escenario inicial.

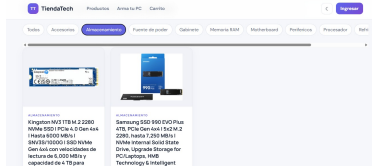
La fragmentación de CockroachDB organiza rangos del motor; la partición JDBC determina cómo Spark solicita datos en paralelo. Los tiempos incluyen planificación, comunicación y materialización. Para reproducir se deben conservar consulta, recursos, número de ejecutores y criterio de finalización.

5.4. Interfaces y capas

Las figuras 5, 6 y 7 documentan rutas públicas y administrativas de la SPA. Las figuras 8 y 9 muestran el recorrido Android y la capacidad de cámara. Estas capturas acreditan presencia y presentación de las funciones observadas; no sustituyen pruebas de aceptación ni demuestran disponibilidad.



(a) Página principal.

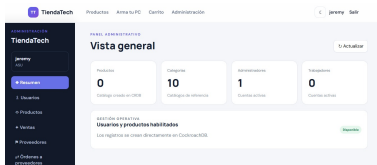


(b) Catálogo.

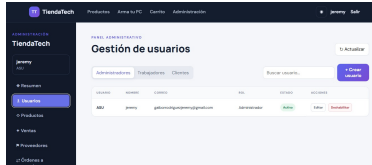


(c) Armado asistido.

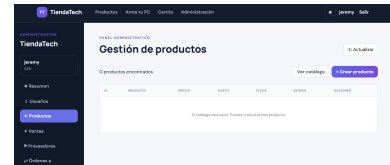
Figura 5: Rutas públicas de la aplicación web TiendaTech. Evidencia propia del equipo.



(a) Resumen administrativo.

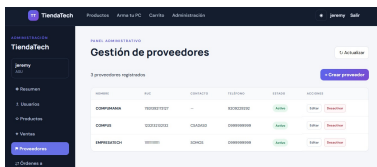


(b) Usuarios.

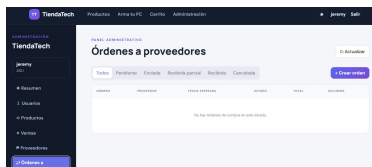


(c) Productos.

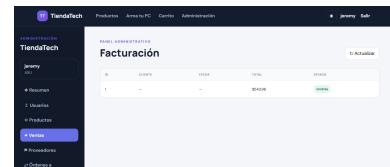
Figura 6: Gestión administrativa de resumen, usuarios y productos. Evidencia propia del equipo.



(a) Proveedores.



(b) Órdenes a proveedores.

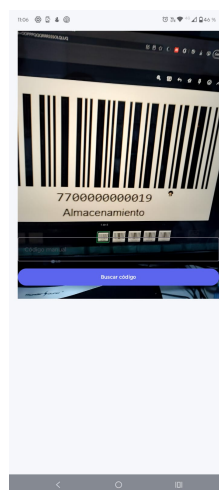


(c) Facturación.

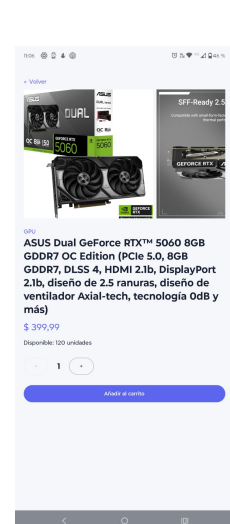
Figura 7: Gestión administrativa de proveedores, órdenes y facturación. Evidencia propia del equipo.



(a) Inicio de sesión.

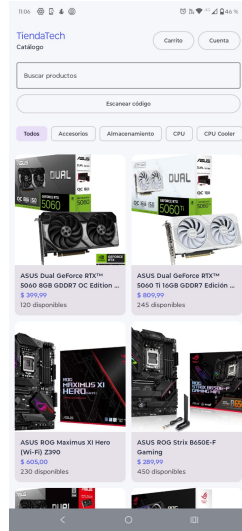


(b) Escaneo de código.

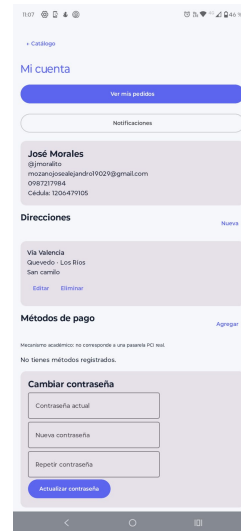


(c) Detalle resuelto.

Figura 8: Autenticación y cámara en la aplicación Android. Evidencia propia del equipo.



(a) Catálogo y búsqueda.



(b) Navegación del catálogo.

Figura 9: Consulta del catálogo desde la aplicación Android. Evidencia propia del equipo.

La cámara se observa en la figura 8; la segunda capacidad, las notificaciones nativas, se sustenta con su implementación y prueba instrumentada, sin exponer datos personales en una captura. La separación de capas permite probar reglas sin desplegar la interfaz ni el motor de datos. Persisten observaciones sobre DTO y reutilización de lógica de órdenes; el refactor no ha eliminado todo acoplamiento.

5.5. Instrumentación

El extracto 2 identifica las unidades medidas. La memoria no es el consumo residente del contenedor, y los segundos de CPU no son un porcentaje.

```
tracemalloc.start()
cpu_0 = time.process_time()
t0 = time.perf_counter()
with ThreadPoolExecutor(max_workers=concurrency) as pool:
    rows = list(pool.map(lab.purchase, cases))
duracion = time.perf_counter() - t0
cpu = time.process_time() - cpu_0
_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
```

Listing 2: Instrumentación del ejecutor del Paso 8.

La reproducción debe partir del hash indicado, no de una mezcla de archivos locales y remotos. Compilar esta memoria no ejecuta experimentos ni modifica la base.

5.6. Trazabilidad de los temas de la asignatura

La tabla 5 reúne tema, archivo, líneas y alcance. Todas las localizaciones se revalidaron en el commit de cierre d7a00cc; los enlaces fijan su hash completo y reflejan las evidencias

integradas hasta el 4 de septiembre. Las líneas de la memoria apuntan a ese corte estable, anterior únicamente a esta actualización de la propia tabla. El inventario editable se conserva en `cierre/trazabilidad-temas.csv`. Una ubicación demuestra dónde se desarrolla el tema; no convierte configuración, decisiones o modelos locales en validación experimental.

Tabla 5: Trazabilidad unificada de temas, fuentes y límites de evidencia.

Tema	Archivo y líneas (enlace al commit)	Alcance comprobable
Microservicios y C4	docs/adr/ADR-009-arquitectura-microservicios.md Líneas 7–33	Decisión y fronteras; vistas C4 en Diseño del sistema.
Capas y SOLID	services/armado-ia/app/domain/catalogo_provider.py Líneas 1–20	Puerto segregado; inversión de dependencias documentada en ADR-008.
Separación de aplicación y persistencia	services/productos-service/src/main/java/com/tiendatech/productos/application/ProductoService.java Líneas 1–35	Servicio dependiente del puerto de dominio; no equivale a validar todo SOLID.
GoF: Strategy	services/armado-ia/app/explicacion/service.py Líneas 19–53	Selección de explicación y fallback bajo una interfaz común.
GoF: Adapter	services/armado-ia/app/explicacion/bedrock_client.py Líneas 32–51	Adapta el SDK a la interfaz de explicación.
GoF: Decorator	Apps/web/frontend/src/main/java/com/tiendatech/frontend/security/JwtGatewayFilter.java Líneas 149–185	Envuelve la petición y expone cabeceras verificadas.
GoF: cadena y método plantilla	services/armado-ia/app/explicacion/validation.py Líneas 31–82	Cinco patrones GoF con los tres anteriores; Repository no se cuenta como GoF.
HTTP y contrato OpenAPI	docs/api/productos.yaml Líneas 1–40	Contrato REST; rutas heredadas pendientes en issue 77.
RPC y gRPC	contracts/stock_reservation.proto Líneas 1–31	Contrato de reservas; no acredita streaming de alertas.
Orden causal: Lamport	services/inventario-service/src/main/java/com/tiendatech/inventario/application/reservation/LamportClock.java Líneas 1–18	Reloj lógico; no aporta exclusión mutua ni consenso.
Fallos, timeouts y Circuit Breaker	services/ventas-service/src/main/java/com/tiendatech/ventas/infrastructure/client/InventarioClient.java Líneas 17–65	Control de llamadas remotas; idempotencia entre servicios aún pendiente.
Propiedad de datos por servicio	docs/db/migrations/V006__eliminar_fk_s_entre_esquemas.sql Líneas 1–24	Migración de referencias entre esquemas y consulta de comprobación.
Fragmentación horizontal	docs/adr/ADR-003-fragmentacion-pedidos.md Líneas 16–34	Rangos temporales de pedidos, índice por usuario; distingue SPLIT de particiones Enterprise.

Tema	Archivo y líneas (enlace al commit)	Alcance comprobable
Fragmentación del catálogo	docs/adr/ADR-012-fragmentacion-catalogo.md Líneas 21–55	Decisión y álgebra por categoría sobre índice; conserva la clave primaria.
Réplica, consenso y quórum	docs/adr/ADR-004-consenso-raft.md Líneas 9–35	Raft del motor y mayoría; no se implementa consenso propio.
CAP por agregado	docs/entrega4/PFC4.tex Líneas 210–223	Política de inventario, pedidos, catálogo y analítica; la campaña desplegada no produjo confirmaciones suficientes para validarla.
Tolerancia a fallos del clúster	docs/evidencias/probar-tolerancia-fallos-e4.ps1 Líneas 1–35	Procedimiento de caída/reintegración; resultados en Persistencia y tolerancia a fallos.
Procesamiento distribuido: Spark	spark/pipeline.py Líneas 185–244	Transformaciones, filtros, joins y agregaciones; resultados separados por lectura.
Amdahl y escalabilidad	spark/analizar_experimento.py Líneas 275–308	Curva teórica frente a speedup observado; no se supone aceleración.
Coordinación 2PC y Saga	experiments/paso7/coordination_lab.py Líneas 63–164	Modelo SQLite con candado global; no son coordinadores distribuidos reales.
Oráculo e invariantes	experiments/paso7/coordination_lab.py Líneas 167–186	Consultas finales; no prueban unicidad estricta ni todos los estados pendientes.
Diseño y estadística experimental	experiments/paso8/run_paso8.py Líneas 94–123	U aproximada y A12; límites exactos discutidos en Amenazas a la validez.
Pruebas y cobertura	docs/experimentos/resultados/iso25010/cobertura-summary.csv Líneas 1–8	Resumen de alcance parcial; Python figura pendiente de XML en este corte.
CI/CD y quality gates	.github/workflows/ci-cd.yml Líneas 17–77	Pruebas y conservación de reportes; configuración distinta de evidencia de ejecución.
Demostración rojo/verde	docs/evidencias/paso9-ci-rojo-verde.md Líneas 1–19	Ejecuciones concretas en rojo y verde, con commits, resultados y efecto colateral documentados.
Observabilidad	ops/observability/prometheus.yml Líneas 1–19	Panel bajo carga y trazas distribuidas exportadas; su alcance y condiciones están documentados en las evidencias del Paso 10.
Seguridad y JWT	Apps/web/frontend/src/main/java/com/tiendatech/frontend/security/JwtGatewayFilter.java Líneas 90–116	Validación y propagación de identidad en gateway; no prueba seguridad integral.

6. Diseño del estudio

6.1. Preguntas de investigación

Se formulan cinco preguntas de investigación para delimitar lo que pueden responder el piloto SQLite y la campaña desplegada del Paso 8. Las dos primeras explicitan factores, población observada y resultados; admiten que la evidencia sea insuficiente o que no exista una diferencia interpretable. Las restantes examinan recuperación, validez del asistente y alcance de reproducción:

1. ¿La campaña de 120 corridas desplegadas, al variar E-2PC/E-Saga, concurrencia nominal y modo de fallo, aporta suficientes compras confirmadas y observaciones persistentes para afirmar conservación de invariantes bajo concurrencia distribuida?
2. ¿La campaña desplegada permite comparar la latencia p95 de compra de E-2PC y E-Saga en las concurrencias 50, 100, 200 y 400 y los modos sin fallo, omisión y temporización?
3. ¿Qué recuperación puede observarse al fallar la pasarela y qué queda fuera del modelo?
4. ¿El banco determinista de compatibilidad valida el asistente real y permite comparar reglas con RAG?
5. ¿Los instrumentos y resultados conservados permiten reproducir estructuralmente la campaña distribuida y sostener sus conclusiones?

Las respuestas conservan esta numeración en la sección de conclusiones. Esta formulación explicita el alcance de datos ya disponibles; no se presenta como un protocolo preespecificado antes de la corrida histórica. Se añaden dos estudios acumulados: escalabilidad analítica y comportamiento del clúster. No se combinan estadísticamente con el laboratorio de coordinación.

6.2. Factores, unidad experimental y procedimiento

La matriz correctiva contiene dos estrategias, concurrencias 50, 100, 200 y 400, y modos **none**, **omission**, **timing**. Cada una de las 24 condiciones tiene cinco repeticiones. La unidad resumida es la ejecución de cinco minutos de medición, precedida por sesenta segundos de calentamiento descartado. Antes de la matriz se aprobaron pilotos basales para ambas estrategias y una rampa sin fallos de 1, 5, 10, 25 y 50 usuarios. La carga utiliza 400 compradores sintéticos y 29 productos con reposición entre fases.

El ejecutor verifica salud de los seis servicios y CockroachDB, conserva una huella estable del banco de 400 casos, repone y redistribuye inventario, y reinicia los servicios entre corridas. La escritura incremental permite reanudar sin duplicar claves. El orden de las condiciones permanece fijo y las ejecuciones comparten anfitrión, por lo que la comparación se trata como no emparejada y su inferencia se limita al entorno observado.

Tabla 6: Protocolo solicitado y configuración realmente ejecutada.

Aspecto	Referencia de la guía	Evidencia del corte
Repeticiones	Cinco por condición	Cinco; 120 ejecuciones.
Retardo	Cinco segundos	Cinco segundos en las 120 filas.
Calentamiento	Sesenta segundos	Sesenta segundos de tráfico descartado.
Duración	Cinco minutos de medición	Trescientos segundos medidos por corrida.
Preflight	Flujo basal saludable	Pilotos 2PC/Saga y rampa 1–50 aprobados.
Coordinación	Participantes distribuidos	Gateway, Pedidos, Ventas, Inventario y CockroachDB.
CPU y memoria	Porcentaje y MiB operativos	Métricas del proceso Locust; contenedores no aislados.

La tabla 6 describe la campaña correctiva publicada. El banco SQLite se conserva como piloto histórico separado y no respalda los resultados distribuidos.

6.3. Variables e invariantes

Se miden percentiles de duración de compra, confirmaciones por segundo, abortos, violaciones del oráculo, tiempo hasta compensación, CPU y memoria rastreada. Aunque algunas columnas se denominan `latencia_pago`, el código obtiene el tiempo de `lab.purchase`; se interpreta como compra local y no como pago aislado.

El oráculo retrospectivo fija una instantánea MVCC de CockroachDB y asigna las órdenes a las ventanas oficiales según su fecha de creación. Contrasta orden, factura, líneas y movimientos de inventario. No existe un ledger independiente de cobros y los estados de checkout completado o fallido estaban en un buffer en memoria perdido durante los reinicios; por ello la captura bancaria y la compensación de intentos cancelados no son verificables. La convergencia Saga mide el outbox factura-inventario y no equivale a reconstruir una compensación cancelada.

$$R = \frac{N_{\text{confirmadas}}}{t_{\text{fin}} - t_{\text{inicio}}}, \quad r_a = \frac{N_{\text{abortadas}}}{N_{\text{operaciones}}}. \quad (2)$$

La ecuación 2 explicita denominadores. Sumar violaciones de distintas consultas y dividir por operaciones no siempre produce una proporción binomial: una operación podría violar más de una regla. Los intervalos publicados de inconsistencia se interpretan con esa reserva.

6.4. Análisis estadístico

Para cada combinación de fallo y concurrencia se comparan los cinco p95 de 2PC con los cinco de Saga. Se informa mediana, Mann–Whitney U, prueba bilateral exacta por

permutación que conserva los empates y tamaño de efecto:

$$\hat{A}_{12} = \frac{\sum_{i=1}^{n_1} \sum_{j=1}^{n_2} [\mathbb{I}(x_i > y_j) + \frac{1}{2}\mathbb{I}(x_i = y_j)]}{n_1 n_2}. \quad (3)$$

En 3, x es 2PC y y es Saga; valores mayores que 0,5 indican p95 mayores para 2PC. La familia contiene doce contrastes y aplica $\alpha = 0,05/12 = 0,004167$. Con cinco observaciones por grupo ninguna comparación correctiva resulta significativa. El análisis es reproducible mediante `experiments/paso8/analyze_corrective_comparison.py`; los CSV derivados no sustituyen el archivo crudo.

6.5. Datos y paralelismo

Para Spark se comparan tiempos de la misma tarea y lectura:

$$S(N) = \frac{T(1)}{T(N)}, \quad E(N) = \frac{S(N)}{N}, \quad S_{\text{Amdahl}}(N) = \frac{1}{(1-f) + f/N}. \quad (4)$$

La ecuación 4 usa f como fracción paralelizable [12]. Despejar la parte no escalable $q = (1/S - 1/N)/(1 - 1/N)$ puede producir valores superiores a uno cuando hay desaceleración: no son fracciones físicas, sino señal de sobrecostos fuera del modelo simple.

Los ensayos del clúster comparan planes y una secuencia de caída/reintegración, sin repeticiones suficientes para estimar una mejora general. Se conservan observaciones individuales, convirtiendo unidades explícitamente.

7. Resultados

7.1. Coordinación local

La tabla 7 transcribe las 24 condiciones de [experimento_resumen.csv](#). Cada fila resume cinco ejecuciones. La figura 10 representa los p95 por ejecución sin mezclar niveles de concurrencia. No se excluyeron ejecuciones al generar la figura.

Tabla 7: Latencia de compra local y confirmaciones por segundo. IC del 95 % del p95 mediano.

Estrategia	Conc.	Fallo	p95 (ms)	IC (ms)	Órdenes/s
2PC	50	none	371.9	[364.1; 374.0]	123.12
2PC	50	omission	346.7	[331.2; 372.9]	114.57
2PC	50	timing	783.7	[524.4; 876.2]	50.42
2PC	100	none	773.1	[646.7; 988.6]	111.33
2PC	100	omission	714.3	[685.9; 795.3]	112.98
2PC	100	timing	1199.8	[929.7; 1347.3]	69.63
2PC	200	none	1656.8	[1507.7; 1864.5]	108.61
2PC	200	omission	1447.0	[1406.0; 1626.1]	113.99

Estrategia	Conc.	Fallo	p95 (ms)	IC (ms)	Órdenes/s
2PC	200	timing	2621.2	[2288.6; 2771.6]	64.61
2PC	400	none	3401.9	[3158.9; 3636.8]	108.07
2PC	400	omission	3085.3	[2927.9; 3139.0]	107.05
2PC	400	timing	5311.8	[4791.1; 5706.5]	61.14
SAGA	50	none	2017.0	[1876.5; 2175.6]	23.38
SAGA	50	omission	1871.5	[1860.4; 1976.3]	22.26
SAGA	50	timing	2286.0	[2211.4; 2402.9]	18.31
SAGA	100	none	3585.9	[3389.5; 3880.0]	26.37
SAGA	100	omission	3946.0	[3754.5; 4079.4]	20.97
SAGA	100	timing	4752.6	[4603.1; 6325.6]	17.72
SAGA	200	none	8088.5	[7075.8; 8708.8]	23.45
SAGA	200	omission	11740.4	[9903.3; 12212.3]	14.31
SAGA	200	timing	12948.6	[12370.4; 13302.2]	13.02
SAGA	400	none	24712.7	[23577.9; 27393.2]	15.32
SAGA	400	omission	22578.9	[22065.3; 23499.9]	14.97
SAGA	400	timing	25660.3	[24655.9; 26674.3]	13.14

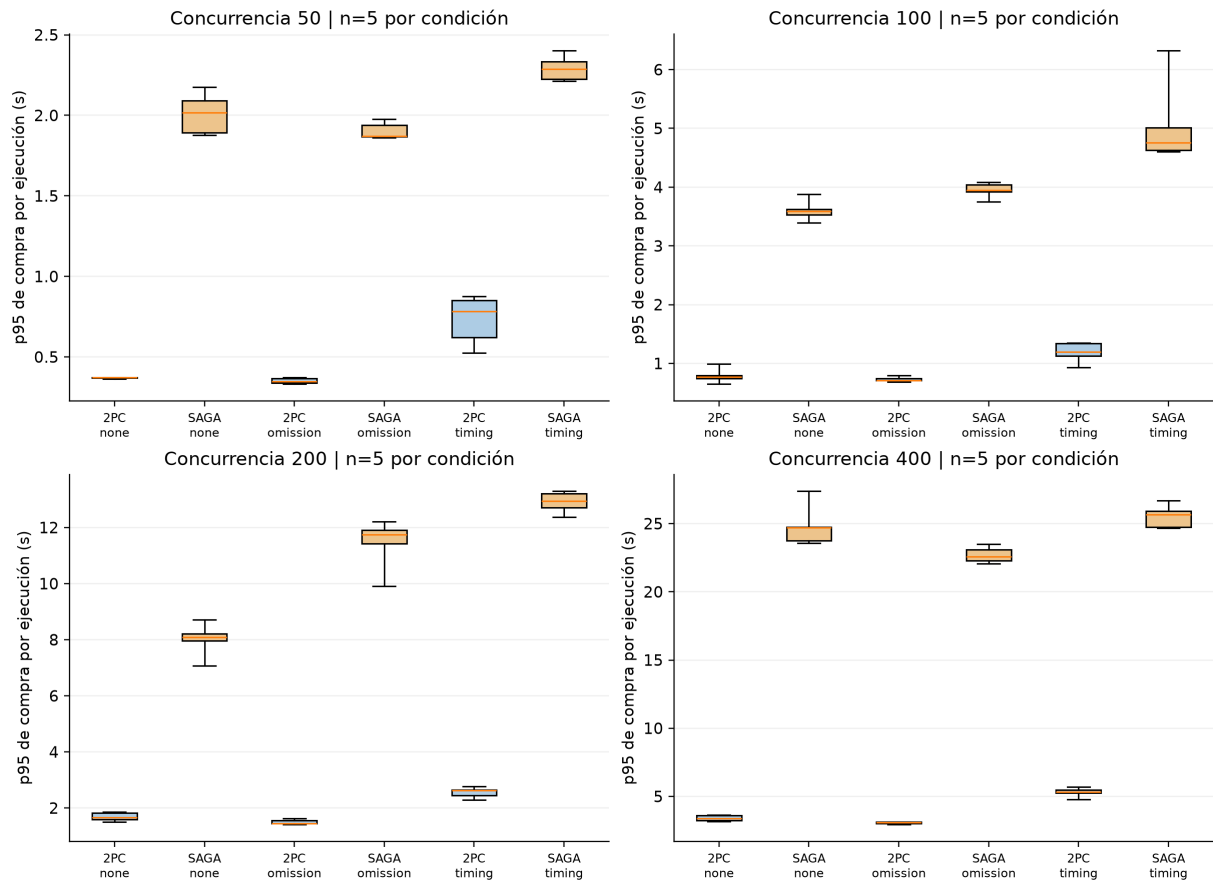


Figura 10: Distribución de p95 de compra por ejecución: cinco repeticiones en cada caja; bigotes mínimo–máximo. Elaboración propia desde el CSV crudo.

Las 120 ejecuciones registran `oracle_pass=True` y cero violaciones en las consultas del oráculo. El informe de compatibilidad registra 120 aciertos sobre 120 casos, cero falsos positivos y cero falsos negativos; el intervalo Wilson de exactitud es $[0,968981; 1]$. Esta evaluación corresponde a reglas locales de socket, RAM y potencia, tal como las implementa el evaluador del experimento.

Las doce comparaciones de p95 registran $U = 0$, $p_{\text{aprox}} = 0,009023$ y $\hat{A}_{12} = 0$. Los datos y resultados de cada contraste están en [comparaciones_mann_whitney.csv](#). La tabla 8 muestra métricas auxiliares de condiciones seleccionadas; no son medidas de recursos de producción.

Tabla 8: Recursos y abortos de condiciones sin fallo, medianas de recursos.

Estrategia	Concurrencia	CPU (s)	Memoria Python (MiB)	Tasa abortos
2PC	50	0.141	0.266	0.0
2PC	400	1.641	2.193	0.0
SAGA	50	0.750	0.266	0.0
SAGA	400	6.078	2.221	0.0

Tabla 9: Concurrencia 400: medianas entre ejecuciones de p50, p99 y tiempo hasta compensación, en ms.

Estrategia	Fallo	p50	p99	Hasta compensación
2PC	none	1799,648	3534,757	—
2PC	omission	1557,841	3252,304	—
2PC	timing	3001,869	5633,613	—
Saga	none	12630,484	25661,211	—
Saga	omission	12141,354	23441,936	21795,425
Saga	timing	13085,967	26835,680	23846,341

La tabla 9 se calcula desde el CSV crudo. El guion indica que no hay evento de compensación; el script guarda cero en esos casos, que no se interpreta como recuperación instantánea. La última columna resume el p95 del tiempo acumulado hasta el evento, calculado en cada ejecución.

7.2. Procesamiento analítico

La tabla 10 recoge medias de T1 en [tiempos_resumen.csv](#). La lectura particionada tiene tres repeticiones y las demás cinco. La figura 11 visualiza esas observaciones agregadas, no una simulación.

Tabla 10: Tiempo de T1 por motor y modalidad.

Motor / lectura	Ejecutores	Repeticiones	Media (s)
Pandas / SQL directo	—	5	2,847172
Spark / JDBC sin partición	1	5	14,557712
Spark / JDBC sin partición	2	5	17,115246
Spark / JDBC sin partición	4	5	19,132645
Spark / JDBC particionado	1	3	17,966838
Spark / JDBC particionado	2	3	16,799444
Spark / JDBC particionado	4	3	17,175005

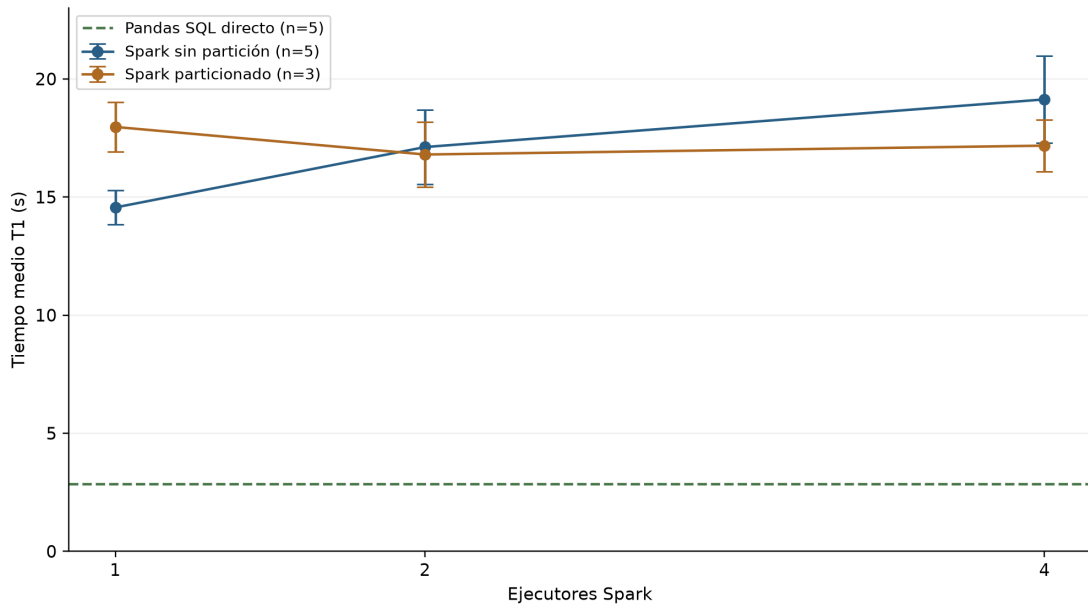


Figura 11: T1: media e intervalo del 95 % publicados, por modalidad y ejecutores. Elaboración propia.

Para Spark sin particionamiento, $S(4) = 0,760884$ y $E(4) = 0,190221$, calculados a partir de las medias. T5 registra 3,526321 s para Pandas y 33,166240, 35,589197 y 37,834008 s para Spark con uno, dos y cuatro ejecutores, respectivamente.

7.3. Persistencia y tolerancia a fallos

La comparación de planes conserva los tiempos individuales de la tabla 11. No se calculan intervalos al no disponer aquí de repeticiones homogéneas de esas consultas.

Tabla 11: Tiempos registrados en comparación de planes, normalizados a milisegundos.

Consulta	Clúster 3 nodos	Nodo único
Historial de usuario	3	3
Top 10 trimestral	625	1100
Cabecera y detalle	11	78
Catálogo	44	2000
Reporte trimestral	4000	8000

En el ensayo de fallo, las consultas tardaron 464,01 ms antes de la caída, 4919,55 ms inmediatamente después, 428,94 ms después de treinta segundos y 918,4 ms tras la reintegración. Las cuatro conservaron el resultado 1644 y estado satisfactorio. El tiempo registrado de reintegración fue 3,62 s. Las rutas de los ficheros originales se conservan en el inventario de cierre.

7.4. Actualización de evidencia del 3 de septiembre de 2026

La revisión del commit [dbe0d77](#) incorpora un piloto posterior al corte de los resultados anteriores. No reemplaza las 120 corridas históricas del Paso 8. Se conservaron copias exactas del CSV, los dos informes del oráculo y las dos bases SQLite en `cierre/actualizacion-20260903/`, con procedencia y sumas SHA-256. La comprobación documental consultó esas bases en modo de solo lectura; no ejecutó una campaña nueva.

El código revisado elimina el candado Python compartido y añade la comprobación `stock_cuadra_con_movimientos`. El piloto contiene 120 operaciones por estrategia, 240 en total, con doce trabajadores, stock inicial 500, semilla 2026, probabilidad de fallo 0,35 y demora 0,01 s según el comando registrado en el README del Paso 7. Son operaciones de un piloto, no 240 repeticiones independientes de la matriz experimental.

Tabla 12: Comprobación del stock en las bases del piloto posterior.

Implementación	Stock inicial	Stock final	Inicial más movimientos
E-2PC	500	319	319
E-Saga	500	490	319

La tabla 12 reproduce una consulta sobre inventario y suma de movimientos. El informe de E-2PC aprueba sus cinco comprobaciones; E-Saga falla la nueva comprobación con una fila de inventario discordante. Esa unidad de conteo no equivale a una compra fallida ni permite calcular una tasa de inconsistencias por operación. Los otros cuatro controles pasan en ambos informes. El hallazgo muestra una anomalía del estado final de esta implementación de Saga y concuerda con el riesgo de actualización perdida de su lectura seguida de escritura; no establece que toda Saga tenga ese comportamiento. E-2PC conserva `BEGIN IMMEDIATE` sobre SQLite: retirar el candado Python no introduce por sí solo participantes distribuidos ni recuperación durable de un coordinador.

También se inspeccionaron las correcciones del ejecutor: doce repeticiones por condición, demora de cinco segundos, calentamiento mediante carga sobre una base separada y cálculo U exacto sin empates con ajuste de Bonferroni. El README remoto identifica la matriz de 288 corridas como pendiente; sus CSV y metadatos históricos no cambiaron respecto del commit evaluado. Por ello, los percentiles y contrastes anteriores siguen siendo antecedentes, no resultados del banco corregido.

Los commits posteriores añaden propagación de `X-Trace-Id`, observaciones recientes de compras, consulta de salud e inyección de fallos, junto con `experiments/paso8/run_microservices.py`. Son avances de implementación. El registro de observaciones conserva como máximo 200 entradas en memoria; el ejecutor mide respuestas HTTP y latencia, y su calentamiento consulta salud. Ni ese registro ni un HTTP exitoso sustituyen una traza distribuida exportada o un oráculo sobre datos persistidos. La etiqueta `COORD` declara configuración y debe verificarse contra la selección efectiva del protocolo. En los cambios inspeccionados no aparece una nueva campaña de microservicios, captura bajo carga ni comprobación de arranque limpio. Estos entregables siguen pendientes.

8. Discusión

8.1. Qué permite concluir la comparación

El banco SQLite histórico mostró p95 menores para E-2PC que para E-Saga, pero ese resultado pertenece a implementaciones locales serializadas y no permite concluir que 2PC sea generalmente superior. La campaña correctiva sí separa procesos, utiliza CockroachDB y produjo una muestra abundante: Saga confirmó 16220 de 25576 checkouts (63,4 %) y 2PC 14055 de 24210 (58,1 %). En el agregado por concurrencia, Saga registra mayor confirmación a 100, 200 y 400 usuarios, mientras que 2PC es ligeramente mayor a 50. Estas proporciones son descriptivas y mezclan los tres modos de fallo.

El oráculo retrospectivo complementa el CSV con una instantánea MVCC de CockroachDB. Entre 43168 órdenes persistidas encontró 6290 sin factura y 13210 con movimiento de inventario ausente o distinto; 104 de las 120 corridas presentaron al menos una violación observable. No encontró importes o líneas de factura discordantes, descuentos duplicados ni stock negativo. Estos ceros se limitan a la persistencia consultada: no prueban captura bancaria ni compensación completa, porque esos estados no se conservaron de forma durable. La política CAP continúa sin validación experimental al no existir una partición de red controlada.

La compatibilidad perfecta describe un evaluador local que replica reglas usadas por el banco sintético. No invoca el servicio real de armado ni un recuperador/generador RAG. Por tanto, no puede justificar precisión del asistente ni una ventaja de IA. La validación futura debe separar generador de etiquetas y sistema evaluado, incluir incompatibilidades de catálogo y registrar respuestas reales sobre un conjunto reservado.

La literatura específica refuerza la separación entre mecanismo y medición. El fallo del coordinador analizado por Gray y Lamport [8] no está representado por la inyección actual; la recuperación por compensación de Garcia-Molina y Salem [7] tampoco queda validada por conteos agregados. Las evaluaciones de tecnologías de saga [9], [10] motivan observar

recuperación y operación junto con latencia. El resultado correctivo permite comparar comportamiento observado, pero no elegir un protocolo productivo con garantía causal o de invariantes.

8.2. Experimento real de checkout bajo carga (campaña correctiva, 2026-09-05)

La campaña correctiva ejecutó 24 condiciones (2 estrategias $\times$ 4 concurrencias $\times$ 3 modos de fallo) contra Gateway, Pedidos, Ventas, Inventario y CockroachDB. Antes de iniciar aprobó el preflight de servicios y base, pilotos basales 2PC/Saga y una rampa sin fallos de 1, 5, 10, 25 y 50 usuarios. Las 120 corridas contienen cinco repeticiones, 60 s de calentamiento descartado y 300 s de medición; todas alcanzaron la concurrencia objetivo y ninguna quedó sin solicitudes o confirmaciones. El banco estable incluye 400 compradores sintéticos y 29 productos redistribuidos para reducir una fila caliente artificial.

Se registraron 208003 solicitudes y 49786 intentos de checkout: 30275 confirmados y 19511 fallidos, para 60,81 % de confirmación global. La tabla 13 agrega los tres modos de fallo y quince corridas por combinación de estrategia y concurrencia. La confirmación cae al aumentar la carga; los errores HTTP 0, 500 y 503, circuit breakers y timeouts a 200 y 400 usuarios se conservan como comportamiento de saturación, no como corridas ausentes.

Tabla 13: Campaña correctiva agregada por estrategia y concurrencia.

Estrategia	Usuarios	Checkouts	Confirmados	Fallidos	Confirmación (%)
2PC	50	5664	5364	300	94,70
2PC	100	6064	5457	607	89,99
2PC	200	7594	2736	4858	36,03
2PC	400	4888	498	4390	10,19
Saga	50	5989	5599	390	93,49
Saga	100	6580	6183	397	93,97
Saga	200	8015	3867	4148	48,25
Saga	400	4992	571	4421	11,44

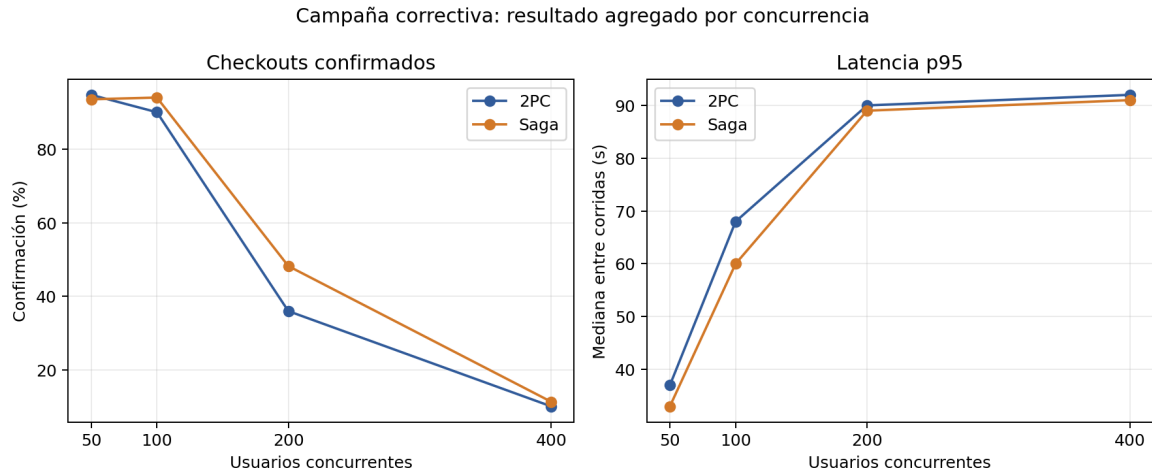


Figura 12: Tasa de confirmación y mediana de p95 entre las quince corridas de cada estrategia y concurrencia. Fuente: CSV correctivo derivado sin alterar los datos crudos.

Tabla 14: Oráculo retrospectivo sobre la instantánea de CockroachDB.

Comprobación persistente	Resultado
Órdenes persistidas en ventanas oficiales	43168
Órdenes sin factura	6290
Importes de factura distintos	0
Líneas de factura distintas	0
Movimiento de inventario ausente o distinto	13210
Descuentos duplicados	0
Eventos de stock negativo	0
Corridas con violación observable	104 de 120

El oráculo usó una instantánea de solo lectura y no alteró el CSV crudo. La tabla 14 separa los hallazgos persistentes de los estados no conservados: 16 corridas quedan `no_verificado`, nunca aprobadas por ausencia de datos.

En las doce comparaciones condicionadas, la mediana p95 de Saga es menor en diez, igual en una y mayor en una. Los valores exactos por permutación están entre 0,015873 y 1; ninguno cruza el umbral Bonferroni 0,004167. La figura 12 permite describir una ventaja observada de Saga en confirmación a carga media y p95 en la mayoría de condiciones, sin afirmar superioridad poblacional. Los archivos `comparacion_estrategias.csv`, `resumen_coord_concurrencia.csv` y `resumen_comparativo.json` conservan el cálculo completo.

8.3. Paralelismo y motor de datos

La desaceleración de T1 al añadir ejecutores es compatible con sobrecostes y entrada poco paralela. Es una interpretación, no un perfil causal exhaustivo. La variante JDBC particionada cambia el acceso y muestra un patrón diferente, pero no supera automáticamente el

tiempo de Pandas. Para este tamaño y entorno, la complejidad operativa de Spark no se justifica por una mejora de T1 observada.

Aplicar Amdahl como ajuste mecánico produciría una fracción no escalable superior a uno para la serie sin partición. Eso no significa que exista más de un cien por ciento de trabajo serial: el modelo omite sobrecostos dependientes del número de ejecutores. Gustafson plantea otra pregunta para cargas crecientes [13]; no corrige retrospectivamente el resultado de carga fija.

La caída de un nodo mostró un pico de latencia con resultado conservado. Es evidencia puntual de continuidad del clúster en ese escenario, no disponibilidad sostenida. El ensayo comparte anfitrión y no cubre partición prolongada ni corrupción. Los planes con tiempos menores en clúster tampoco aíslan por sí solos diferencias de caché, índices y recursos.

8.4. Decisiones de cierre

El cierre mantiene arquitectura y evidencias existentes, y registra deuda en lugar de modificar apresuradamente el producto. Las prioridades son idempotencia y validación de contratos, luego pruebas de recuperación y una medición oficial con recursos y tráfico definidos. La revisión bibliográfica se corrigió porque un DOI existente puede apuntar a un trabajo distinto: comprobar que el enlace abre no basta; debe coincidir con título y autores.

El producto posee instrumentos de calidad y observabilidad antes ausentes. Sin embargo, instalar Prometheus o aprobar CI no acredita un objetivo operacional. La memoria distingue esas capas para que el lector pueda reconocer el avance sin asumir garantías todavía no medidas.

9. Amenazas a la validez

La descripción histórica del candado se refiere al banco SQLite original. El piloto posterior de la sección 7.4 elimina el candado de Python y detecta una discordancia de stock. La evaluación distribuida correctiva ejecutó 120 corridas contra microservicios y CockroachDB y ya no presenta escasez de confirmaciones. El oráculo retrospectivo recupera relaciones persistentes de orden, factura e inventario, pero no la captura bancaria ni los estados de compensación guardados solo en memoria. Persisten además el anfitrión compartido, el orden fijo, la saturación a 200–400 usuarios y la falta de una partición de red controlada.

9.1. Validez interna

Contención del anfitrión. Todas las ejecuciones comparten máquina y pueden sufrir carga externa. Mitigación: conservar repeticiones y recursos, aislar futuras corridas y aleatorizar orden. No se supone aislamiento retroactivo.

Candado global y serialización. La versión vigente de `experiments/paso7/coordination_lab.py` ya no crea `self.db_lock`; el piloto conserva el escritor único de SQLite y queda como demostración local. La evidencia principal es `experiments/paso8/result`

ados-reales/correctiva-20260905-final-v2/, ejecutada mediante Gateway, Pedidos, Ventas, Inventario y CockroachDB. Sus 30275 confirmaciones permiten analizar el flujo real; el oráculo retrospectivo añade evidencia persistente parcial y detecta 13210 inconsistencias de inventario, sin reconstruir pagos ni compensaciones completas.

Semillas y orden. El cálculo de semillas varía entre estrategias y las condiciones se recorren en orden fijo. Mitigación: utilizar listas idénticas de operaciones/fallos y orden aleatorio balanceado en una réplica. Las comparaciones actuales son no emparejadas.

Configuración y preparación. La campaña correctiva registra cinco segundos de demora, sesenta de calentamiento y trescientos de medición en las 120 filas. Los pilotos y la rampa reducen el riesgo de comenzar con un flujo basal roto. Permanece el riesgo de cambios térmicos y carga externa durante 22,54 horas en un solo anfitrión.

Saturación a carga alta. La tasa de confirmación agregada cae de 94,1 % con 50 usuarios a 10,8 % con 400. Esta caída es un resultado del entorno y no invalida las corridas, pero limita la estimación de latencia del trabajo completado porque aumenta la censura por timeout. Se reportan confirmaciones y fallos junto con cada percentil.

9.2. Validez externa

Carga sintética. Los 400 compradores y 29 productos redistribuidos reducen la concentración artificial previa, pero no representan navegación, usuarios heterogéneos ni demanda real. El alcance se limita al flujo sintético de checkout.

Topología y pasarela. Un anfitrión y una función de pago no modelan latencia geográfica ni fallos bancarios. Mitigación: ensayar red y participantes independientes sin usar credenciales ni pagos reales; no extrapolar los percentiles a producción.

Catálogo artificial. Casos generados por reglas próximas al evaluador favorecen resultados perfectos. Mitigación: etiquetas independientes, conjunto reservado y llamadas al asistente real antes de evaluar reglas frente a RAG.

9.3. Validez de constructo

Reconstrucción retrospectiva. La asignación de órdenes usa ventanas temporales y una instantánea MVCC posterior a la campaña. Permite auditar relaciones persistentes, pero 13321 órdenes sintéticas quedaron fuera de las ventanas y no reconstruye causalidad ni estados efímeros. Los 13210 desajustes de inventario son violaciones observables; los ceros de otras consultas no equivalen a riesgo nulo. Una réplica debe persistir identificadores y estados parciales y añadir controles positivos para cada invariante.

Latencia mal rotulada. Una columna de pago mide compra completa. Mitigación aplicada: nombrarla latencia de compra en el informe y conservar el nombre original en datos para trazabilidad.

Oráculo incompleto. No existe ledger de cobros y el buffer de 200 observaciones se perdió entre reinicios; la compensación cancelada queda `no_verificado`. Mitigación: persistir el ciclo completo por identificador y probar que mutaciones deliberadas de cada invariante son detectadas.

Recursos parciales. CPU en segundos y memoria Python no equivalen a porcentaje CPU/RSS. Mitigación aplicada: unidades explícitas; futura instrumentación a nivel de proceso o contenedor.

9.4. Validez de conclusión

Resolución de la prueba y potencia. Con cinco observaciones por grupo, sin empates, existen $\binom{10}{5} = 252$ asignaciones de rangos bajo la hipótesis nula. El menor valor p bilateral exacto de Mann–Whitney es

$$p_{\min} = \frac{2}{\binom{10}{5}} = 0,0079365\dots > \alpha_{\text{Bonferroni}} = \frac{0,05}{12} = 0,0041667\dots \quad (5)$$

La campaña correctiva conserva cinco observaciones por grupo y presenta empates en varios p95. La prueba exacta por permutación incorpora esos empates; sus doce valores p van de 0,015873 a 1 y ninguno cruza Bonferroni. La limitación sigue siendo estructural: con cinco repeticiones no puede obtenerse una conclusión confirmatoria para una familia de doce contrastes. Una réplica futura debe aumentar repeticiones o predefinir una familia menor según la pregunta científica.

Tamaño de efecto heterogéneo. Para p95, $\hat{A}_{12}$ de 2PC frente a Saga varía entre 0,48 y 0,98: el solapamiento cambia por fallo y concurrencia. Esa heterogeneidad descarta trasladar al sistema real la separación perfecta del piloto SQLite y exige interpretar cada condición por separado.

La suma de violaciones no es necesariamente binomial y un cero observado no prueba riesgo nulo. Mitigación: definir una variable binaria por operación para cada invariante y reportar su intervalo por separado. Los ensayos puntuales del clúster tampoco permiten inferir disponibilidad sostenida; se mantienen como observaciones descriptivas.

10. Calidad del producto

10.1. Modelo y criterios

ISO/IEC 25010:2023 organiza nueve características de calidad del producto; ISO/IEC 25023:2016 aporta medidas [21], [22]. Las normas no imponen los umbrales académicos de cobertura o complejidad usados aquí. Esos objetivos proceden de la guía y deben interpretarse según alcance y método.

La tabla 15 distingue evidencia favorable, parcial y ausente. No se declara certificación ISO ni conformidad integral por disponer de un CSV. La seguridad requiere pruebas y configuración coherentes, mientras que la mantenibilidad requiere que el código sea comprensible además de superar un umbral.

Tabla 15: Evaluación por características ISO/IEC 25010:2023.

Característica	Evidencia y límite
Adecuación funcional	Flujos y pruebas implementados; no aceptación completa de todos los recorridos.
Eficiencia	Carga oficial de 50 usuarios/60 s: P95 agregado 610 ms; no cumple el umbral de 500 ms. Una sola corrida no permite estimar un IC95 del P95.
Compatibilidad	Contratos HTTP/gRPC; Pact y ensayo completo entre consumidores no acreditados.
Capacidad de interacción	Interfaces web/Android; sin estudio formal de usuarios.
Fiabilidad	Ventana oficial de 3600 s: 3588/3588 sondeos, 100 % (IC95 % Wilson 99,8931–100 %); en carga, 0/2112 respuestas 5xx (límite superior 0,1816 %).
Seguridad	JWT, restricciones de exposición y pruebas negativas; no auditoría integral.
Mantenibilidad	Cobertura agregada 32,28 % sobre el árbol completo de los ocho servicios medidos (seis Java, el Gateway y armado-ia ; 11,72–78,26 % por módulo; solo el Gateway alcanza el 70 %) y PMD máximo 9; cifra regenerada el 18 de septiembre de 2026 en una corrida de CI que también agrega JaCoCo al Gateway y amplía armado-ia a su paquete app completo, ver §10.2.
Flexibilidad	Contenedores y configuración; no ensayo multientorno exhaustivo.
Seguridad operacional	Riesgos de stock/pago identificados; no análisis completo de daño operacional.

10.2. Cobertura y complejidad

La tabla 16 procede del informe **informe-umbral-70.json**, regenerado el 18 de septiembre de 2026 sobre el árbol completo de cada servicio Java, el paquete **app** completo de **armado-ia** y el Gateway (**Apps/web/frontend**), que hasta entonces no tenía JaCoCo configurado. Las cifras resultantes son más bajas que las reportadas antes de ese ajuste, pero honestas: solo el Gateway alcanza el umbral del 70 %.

Tabla 16: Cobertura en el alcance de los informes disponibles.

Módulo	Cubiertas	Medidas	Porcentaje
Inventario	311	1396	22,28
Órdenes a proveedores	64	546	11,72
Pedidos	443	1938	22,86
Productos	116	512	22,66
Usuarios	442	1037	42,62
Ventas	161	493	32,66
Gateway	162	207	78,26
armado-ia	580	931	62,30
Agregado (ocho servicios)	2279	7060	32,28

El umbral del 70 % se verifica directamente desde los ocho XML mediante [verificar_umbral.py](#). El resultado versionado `informe-umbral-70.json`, regenerado el 18 de septiembre de 2026 sobre una corrida limpia de los ocho servicios, muestra que solo el Gateway alcanza el 70 % (78,26 %): los otros siete quedan entre 11,72 % (`ordenes-proveedores-service`) y 62,30 % (`armado-ia`), con un agregado ponderado de 32,28 % sobre 7060 líneas instrumentables. La comprobación no depende de una publicación externa en Codecov y mide el árbol completo de cada servicio Java, el paquete `app` completo de `armado-ia` (antes restringido a `app.domain` y `app.explicacion`) y, desde esta corrida, el Gateway. El umbral permanece declarado en el goal `check` de JaCoCo de cada `pom.xml`, con `haltOnFailure=false` para no bloquear la integración continua mientras se amplía la suite de pruebas hacia dominio, infraestructura y presentación.

Las cifras de `Usuarios` y `Productos` se corrigieron el 18 de septiembre de 2026: una primera medición local había reportado 511/1037 (49,28 %) y 138/512 (26,95 %) porque la corrida en IntelliJ tenía activo el perfil Maven `pact`, que agrega `src/test/pact-java/` (pensado para correr solo con `-Ppact` en los jobs de verificación de contrato de CI) al `mvn test` plano. La primera corrida en vivo del job `coverage-java` en CI, sin ese perfil, ya midió los valores correctos, confirmados localmente con `mvn clean test` y el perfil `pact` desactivado.

El resumen regenerado en `docs/experimentos/resultados/iso25010/complexidad/summary.csv`, que acompaña al código refactorizado, aplica el mismo criterio al servidor Java y al cliente TypeScript/React: máximo de complejidad ciclomática por método o función estrictamente inferior a diez. PMD 7.17.0 mide Java y la regla `complexity` de ESLint mide la web en su variante clásica; cada herramienta cuenta las decisiones propias de su lenguaje. La ejecución local posterior al refactor volvió a medir los siete módulos Java completos, sin excluir los métodos modificados. Los siete servicios Java y el cliente web cumplen el umbral. No se extiende esta medición al servicio Python. El [resumen inicial anterior al refactor](#) se conserva como referencia histórica y no contiene los valores de la tabla actual.

Módulo	Máximo	Objetivo	Estado
Gateway	7	< 10	Cumple
Inventario	8	< 10	Cumple
Órdenes a proveedores	8	< 10	Cumple
Pedidos	9	< 10	Cumple
Productos	8	< 10	Cumple
Usuarios	9	< 10	Cumple
Ventas	8	< 10	Cumple
Cliente web	9	< 10	Cumple

La medición web analiza el código de producción de **src** (TypeScript y TSX), incluyendo componentes y callbacks; excluye pruebas y herramientas fuera de ese directorio. Se fuerza el umbral de reporte a cero para registrar todas las funciones, incluidas las de complejidad uno, sin permitir supresiones inline. La ejecución local del 13 de septiembre de 2026, después del refactor, registró 434 funciones en 19 archivos de producción y un máximo de 9. Los valores, ubicaciones y versión de ESLint se conservan en `docs/experimentos/resultados/iso25010/complejidad/webapp-complexity.json`, junto al resumen regenerado. La [medición web inicial](#) registraba 404 funciones y un máximo de 26 en `api`; el refactor separó responsabilidades HTTP y secciones de presentación, manteniendo el umbral y el alcance del análisis. El comando `npm run complexity:report` actualiza la fila web del resumen conjunto y devuelve un código de error si el máximo alcanza diez. El job `web-quality` descarga la medición PMD de la misma ejecución de CI, añade la medición web y conserva ambas en el artefacto. La medición local ahora cumple; su resultado no sustituye la confirmación de la siguiente ejecución remota de CI. Las pruebas web se ejecutan también cuando falla la complejidad, para conservar diagnósticos independientes.

El análisis general de TypeScript y React Hooks se mantiene separado mediante `npm run lint:report`, con sus reportes `webapp-eslint.json` y `webapp-eslint-summary.csv`, y sigue fallando ante errores o advertencias. Sus recuentos de incidencias no se usan como sustituto de la complejidad ciclomática.

La cobertura del cliente se publica por separado en el [resumen de cobertura web](#), regenerado el 14 de septiembre de 2026 tras retirar de `vitest.config.ts` la lista de doce vistas seleccionadas a mano: 62 pruebas y 71,96 % de líneas sobre el árbol completo de **src** (antes 19 pruebas y 53,39 % sobre solo tres vistas). Se conservan además sentencias (56,26 %), ramas (45,10 %) y funciones (44,47 %), con servicios HTTP simulados. La ampliación de alcance deja visibles dos huecos que la medición anterior no cubría: `App.tsx`, el punto de entrada de la aplicación, y `src/services/cart.ts`, la lógica de carrito, están en 0 % de cobertura por no haber sido medidos antes. Estos resultados no se agregan a la cobertura del backend ni se someten automáticamente al umbral del 70 % de los ocho XML anteriores.

El 71,96 % de líneas es la cifra menos representativa de las cuatro y no debe leerse como el resumen de la cobertura web: Istanbul cuenta “líneas” por línea física del archivo, y varios componentes de `src/views/` están escritos con muy pocas líneas físicas por mucho código (JSX y lógica condensados), lo que infla ese porcentaje respecto a sentencias y ramas. El propio `coverage-summary.json` lo muestra: `CheckoutView.tsx` reporta 1/1 línea (100 %) pero solo 24/32 sentencias (75 %) y 9/21 ramas (42,85 %); `AccountView.tsx` reporta 2/2 líneas (100 %) frente a 35/40 sentencias (87,5 %) y 28/44 ramas (63,63 %);

`RecoveryView.tsx` y `WorkerView.tsx` reportan 1/1 línea (100 %) con 15 y 11 sentencias reales respectivamente. Sentencias (56,26 %) y ramas (45,10 %) miden sobre unidades de código que no dependen del formato de línea del archivo, así que son las cifras que mejor reflejan cuánto del comportamiento real de la SPA está bajo prueba; el 71,96 % se conserva por ser la métrica que reporta Vitest por defecto, no porque sea la más honesta de las cuatro.

10.3. CI, carga y observabilidad

Se verificó la demostración rojo/verde de GitHub Actions mediante dos ejecuciones concretas de *CI-CD quality gate*: la [ejecución roja](#) concluyó en `failure` sobre el commit `f58869b`, mientras que la [ejecución verde](#) concluyó en `success` sobre `bd027f0`. Ambas fueron disparadas por `pull_request`; los PR `#81`, `#82` y `#83` conservan la secuencia de fallo intencional, reversión y corrección final. Son ejecuciones de CI, no solicitudes de incorporación de cambios. La publicación de imágenes en GitHub Container Registry (GHCR) pertenece a otro workflow, definido en `.github/workflows/publish-images.yml`, y se activa después de que el quality gate de `main` finaliza satisfactoriamente o mediante ejecución manual; la migración está registrada en el commit `a922bf4`.

El registro `docs/evidencias/paso9-ci-rojo-verde.md` concilia cada enlace con su resultado, evento y commit mediante la API de GitHub. También documenta el BOM UTF-8 introducido durante la reversión y su corrección, de modo que el fallo rojo queda separado de ese efecto colateral de compilación. Con esta evidencia versionada, C7 deja de figurar como deuda documental en la tabla 18; la verificación no implica que futuras ejecuciones o publicaciones queden aprobadas automáticamente.

La prueba de carga previa registró 1911 solicitudes y 862 fallos. El diagnóstico relacionaba parte del problema con respuestas de productos y con limitación de solicitudes por IP; no constituía aprobación de rendimiento. Una repetición posterior de la misma prueba oficial de `tests/load/` (50 usuarios, rampa 5/s, 60s), documentada con sus métricas crudas en [carga y panel Grafana](#), registró 2194 solicitudes y 1894 fallos (86,3 %). Se confirmó la causa en el código real de `GatewayTrafficFilter.java` (líneas 41–64): el límite de 300 solicitudes por 60s por IP de origen, activado porque el generador de carga corre desde un único host y por tanto un único par de transporte. Los `.csv` adjuntos confirman que el 100 % de los fallos son 429; ninguno es 5xx. Sigue sin haber una prueba de carga distribuida desde múltiples orígenes que evalúe el sistema sin ese límite por IP.

El repositorio contiene métricas Prometheus, logs JSON, recolección Alloy y configuración de Grafana. La actualización del 4 de septiembre aporta evidencia operativa puntual: se agregó un Grafana local (`docker-compose.yml`) cuyo datasource y dashboard se cargan por provisioning como código (`ops/observability/grafana/provisioning/`), no por importación manual, y se capturó con datos reales de la repetición de carga anterior ([captura del panel](#)). Se aportan además trazas OpenTelemetry separadas de checkout y de incorporación al carrito, con un identificador por operación, incluyendo el canal TCP crudo entre `pedidos-service` e `inventario-service`, que por transportar JSON sobre un Socket plano en vez de HTTP quedaba fuera del alcance de la auto-instrumentación y no aparecía en ninguna traza; se instrumentó manualmente con spans `reservation.tcp.reconcile` y propagación del `traceparent` dentro del propio mensaje ([traza distribuida completa](#)). La disponibilidad de una hora y la tasa de errores 5xx de la corrida oficial ya se midieron

(100 % de disponibilidad sobre 3600s continuos frente al objetivo de 99,5 %, y 0 % de respuestas 5xx sobre 2112 solicitudes): ambas cumplen. Permanece sin medir el p95 productivo bajo múltiples orígenes de carga simultáneos; tampoco se infiere cobertura E2E o de la SPA a partir de pruebas unitarias Java.

La sesión posterior del 4 de septiembre conserva una repetición estable en [el registro de ejecución ISO](#): 2112 solicitudes, cero fallos, 36,45 solicitudes/s y p95 agregado de 610 ms, contrastados con el CSV de `load-final/`. El p95 no alcanza el objetivo menor a 500 ms; cero fallos observados no garantiza fiabilidad general. La disponibilidad de una hora concluyó después de 3600s con 3588 de 3588 muestras exitosas: 100 % frente al objetivo de 99,5 %. Esta sesión ISO y la campaña correctiva responden preguntas distintas y no mezclan sus resultados.

Las figuras 13 y 14 hacen visible la evidencia operativa descrita: la primera corresponde al panel capturado durante la carga conjunta y la segunda a las trazas exportadas en esa sesión. Son fotografías de una ejecución fechada; las métricas crudas y los JSON exportados siguen siendo la fuente auditable.



Figura 13: Dashboard Grafana con datos reales durante la sesión conjunta de carga. Fuente: evidencia propia del Paso 10.

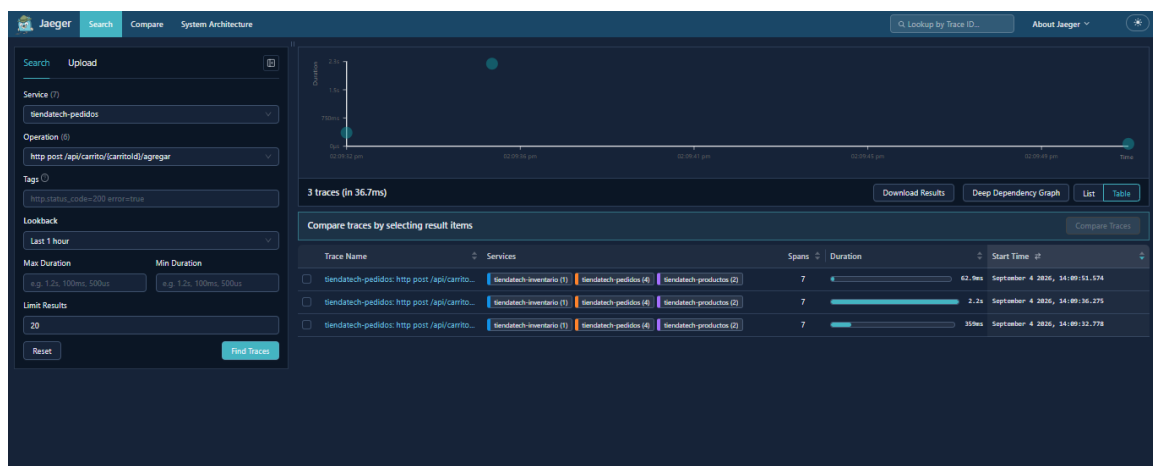


Figura 14: Trazas distribuidas exportadas durante la sesión conjunta; incluyen el recorrido instrumentado y el canal TCP. Fuente: evidencia propia del Paso 10.

El punto 17 de la guía de cierre observó que ninguna captura citada mostraba los cinco paneles del dashboard con series de datos a la vez, que el manuscrito seguía enlazando solo la captura antigua de `paso10-item5-grafana-carga.md` y que no mencionaba la inyección de fallos usada para producir el panel 5xx. Se cerró con evidencia adicional en `punto17-observabilidad-gateway.md`: el Gateway (`Apps/web/frontend`) quedó instrumentado igual que los seis microservicios Java y scrapeado por Prometheus; el eje de los paneles 4xx/5xx tenía la unidad de Grafana mal configurada (la consulta PromQL devuelve una fracción 0–1, no un porcentaje 0–100), lo que explica por qué esos paneles aparecían planos en capturas anteriores del proyecto pese a que el 429 real sí llegaba al Gateway; y, dado que el limitador de tasa nunca deja pasar tráfico suficiente para un 5xx orgánico, se usó el mecanismo de inyección de fallos ya existente en el código (`ExperimentFaultInjector` en `ventas-service`, activado por el encabezado `X-Failure-Mode: omission` y una variable de entorno desactivada por defecto) para producir fallos reales de checkout. Una corrida de carga de 400 usuarios generó 33 006 solicitudes con 3 812 429 reales, y cuatro de cinco checkouts disparados con el inyector de fallos activo devolvieron un 5xx real; las dos capturas resultantes (`dashboard-gateway-traffic-latencia.png` y `dashboard-gateway-5xx-real.png`) muestran, entre ambas, los cinco paneles con series simultáneas y datos reales, incluido el panel 5xx subiendo para pedidos y ventas —los servicios que participan directamente en el checkout con fallo inyectado—, mientras que `gateway` se mantiene cerca de 0 % en ese mismo panel porque su tráfico total, dominado por la carga pública, diluye la misma cantidad absoluta de fallos a una fracción mucho menor; no es ausencia de instrumentación. Se corrigió además un problema real detectado en los `.csv` crudos de esta corrida: Locust, en la máquina Windows donde se generaron, los escribía con doble retorno de carro (`\r\r\n`) en vez de `\r\n`, lo que producía filas vacías intercaladas al leerlos con un lector CSV estándar; se normalizó la salida ya generada y se corrigió `run-load-test.ps1` para que ninguna corrida futura vuelva a quedar así. Por último, se agregó una prueba (`GatewayTrafficFilterTest.registraMetricasPrometheusConNombreYStatusReales`) que construye el filtro del Gateway con un `MeterRegistry` real y falla si se renombra la métrica o se fija el `status` a un valor constante en vez del código de respuesta real —la mutación exacta bajo la cual las pruebas anteriores permanecían en verde. Queda pendiente proteger o justificar formalmente la exposición sin autenticación de `/metrics` y `/health` en el Gateway, y todavía no existe un registro de datos crudos de la corrida de inyección de fallos citada (solo las capturas), aunque el script ya quedó preparado para generarlo en la próxima corrida.

11. Gestión de la revisión cruzada

11.1. Criterio de respuesta

La tabla 17 recoge las incidencias 16–78 recuperadas en el corte: 63 registros, de los cuales 53 figuran cerrados y 10 abiertos. Los issues 40, 46 y 48 fueron contrastados con 41, 47 y 49, recibieron una respuesta con evidencia y se cerraron administrativamente como duplicados el 4 de septiembre de 2026. Los diez abiertos restantes corresponden al trabajo pendiente identificado en el proyecto actual.

AC significa aceptar y corregir según la respuesta publicada; AD significa aceptar y diferir en

este cierre documental; R significa rebatir con justificación. El estado de GitHub y la acción documental son columnas distintas. Las respuestas enlazadas contienen la justificación y, cuando existen, referencias a commits. Un cierre administrativo no constituye por sí solo prueba de ejecución ni invalida los límites de las secciones de calidad y método.

Los abiertos con corrección parcial o pendiente se incorporan como deuda explícita; AD no significa que se haya alterado el proyecto. El registro de los tres duplicados se conserva para evitar contar dos veces una misma corrección, aunque su consolidación administrativa ya quedó realizada.

Tabla 17: Respuesta y evidencia individual de revisión cruzada.

N.º	Observación	Estado	Acción	Evidencia
16	Aplicar framework para el frontend	Cerrado	AC	Respuesta 16
17	Aplicar temas de tareas formativas al proyecto	Cerrado	AC	Respuesta 17
18	Investigación de temas de la materia aplicables al proyecto	Cerrado	AC	Respuesta 18
19	Hacer que usuarios no dañe todo el proyecto	Cerrado	AC	Respuesta 19
20	Primera versión de aplicación móvil	Cerrado	AC	Respuesta 20
21	Primera versión de la IA asistente	Cerrado	AC	Respuesta 21
22	Aplicación móvil finalizada	Cerrado	AC	Respuesta 22
23	Asistente de IA finalizado	Cerrado	AC	Respuesta 23
24	Arreglar estructura del proyecto	Cerrado	AC	Respuesta 24
25	Implementar validación JWT en el API Gateway	Cerrado	AC	Respuesta 25
26	Configurar JWT_SECRET compartido entre Gateway y usuarios-service	Cerrado	AC	Respuesta 26
27	Definir rutas públicas del API Gateway	Cerrado	AC	Respuesta 27
28	Crear patrón reutilizable de validación JWT	Cerrado	AC	Respuesta 28
29	Cerrar exposición pública de los puertos 8081-8086	Cerrado	AC	Respuesta 29
30	Agregar healthcheck a usuarios-service	Cerrado	AC	Respuesta 30
31	Consolidar manejadores globales de excepciones de usuarios-service	Cerrado	AC	Respuesta 31
32	Unificar rutas de creación de usuarios	Cerrado	AC	Respuesta 32
33	Corregir respuesta HTTP al crear usuarios	Cerrado	AC	Respuesta 33
34	Documentar política común de autenticación y autorización	Cerrado	AC	Respuesta 34
35	Verificar integraciones internas con protección JWT	Cerrado	AC	Respuesta 35
36	Incorporar validación JWT en ventas-service y añadir healthcheck	Cerrado	AC	Respuesta 36

N.º	Observación	Estado	Acción	Evidencia
37	Configurar timeouts explícitos en InventarioClient	Cerrado	AC	Respuesta 37
38	Implementar el patrón outbox al generar una factura	Cerrado	AC	Respuesta 38
39	Guardar la factura y el evento outbox dentro de la misma transacción de base de datos	Cerrado	AC	Respuesta 39
40	Crear el proceso que reintente eventos pendientes y marque un evento como procesado solo después de la confirmación de inventario	Cerrado	AC	Respuesta 40
41	Crear el proceso que reintente eventos pendientes y marque un evento como procesado solo después de la confirmación de inventario	Cerrado	AC	Respuesta 41
42	Añadir Circuit Breaker y reintentos seguros alrededor de la comunicación con inventario	Cerrado	AC	Respuesta 42
43	Coordinar con José la idempotencia del descuento de existencias	Abierto	AD	Respuesta 43
44	Reemplazar Map String,Integer por GenerarFacturaRequest y aplicar @Valid.	Cerrado	AC	Respuesta 44
45	Crear DTO de salida para facturas y dejar de exponer directamente la entidad JPA	Cerrado	AC	Respuesta 45
46	Incorporar validación JWT en ordenes-proveedores-service. y añadir healthcheck	Cerrado	AC	Respuesta 46
47	Incorporar validación JWT en ordenes-proveedores-service y añadir healthcheck	Cerrado	AC	Respuesta 47
48	Configurar timeouts explícitos en InventarioClient	Cerrado	AC	Respuesta 48
49	Configurar timeouts explícitos en InventarioClient	Cerrado	AC	Respuesta 49
50	Añadir Circuit Breaker y reintentos seguros para la comunicación con inventario	Cerrado	AC	Respuesta 50
51	Crear OrdenCompraResponseDTO y ProveedorResponseDTO	Cerrado	AC	Respuesta 51
52	Verificar que las transiciones enviar y cancelar sean idempotentes o rechacen correctamente las repeticiones	Cerrado	AC	Respuesta 52
53	Si se aprueba gRPC: implementar el cliente suscriptor de alertas de inventario y evitar órdenes duplicadas ante reconexiones o mensajes repetidos	Abierto	AD	Respuesta 53
54	Documentar por qué /enviar y /cancelar representan transiciones de negocio en lugar de un PATCH genérico	Cerrado	AC	Respuesta 54

N.º	Observación	Estado	Acción	Evidencia
55	Incorporar validación JWT en pedidos-service	Cerrado	AC	Respuesta 55
56	Reutilizar OrdenCompraService como punto único de lógica al recibir alertas	Abierto	AD	Respuesta 56
57	Añadir healthcheck	Cerrado	AC	Respuesta 57
58	Configurar connect timeout y read timeout en FacturaClient, ProductoClient y UsuarioClient	Cerrado	AC	Respuesta 58
59	Crear un ApiExceptionHandler global para respuestas de error uniformes	Cerrado	AC	Respuesta 59
60	Verificar que cualquier reintento de una operación de escritura sea idempotente o utilice una clave contra duplicados	Cerrado	AC	Respuesta 60
61	Añadir Circuit Breaker y reintentos controlados en los clientes internos donde sean seguros	Cerrado	AC	Respuesta 61
62	Cambiar operaciones void por respuestas HTTP explícitas, normalmente 204 No Content	Cerrado	AC	Respuesta 62
63	Corregir códigos HTTP de creación	Cerrado	AC	Respuesta 63
64	Reemplazar mapas y entidades expuestas por DTO	Abierto	AD	Respuesta 64
65	Añadir un estado formal del pedido solo si se implementará seguimiento en tiempo real	Cerrado	R	Respuesta 65
66	Mantener TCP/WebSocket como funcionalidad separada de las correcciones urgentes	Cerrado	AC	Respuesta 66
67	Validación JWT para productos e inventario.	Cerrado	AC	Respuesta 67
68	Healthcheck para productos e inventario.	Cerrado	AC	Respuesta 68
69	Idempotencia para inventario	Cerrado	AC	Respuesta 69
70	Respuesta ante eventos repetidos	Cerrado	AC	Respuesta 70
71	Mejora de productos para respuestas incorrectas	Cerrado	AC	Respuesta 71
72	Revisión de códigos HTTP en productos	Cerrado	AC	Respuesta 72
73	Preparación de inventario como servidor	Abierto	AD	Respuesta 73
74	Creación de contrato .proto	Abierto	AD	Respuesta 74
75	Pruebas de conexión	Abierto	AD	Respuesta 75
76	Exponer entidades	Abierto	AD	Respuesta 76
77	Revisión de rutas	Abierto	AD	Respuesta 77
78	Telemetría UDP	Abierto	AD	Respuesta 78

11.2. Justificación y coste de la deuda

La idempotencia del receptor no garantiza que ventas y proveedores propaguen una misma clave en cada reintento. Por eso el issue 43 permanece abierto aunque 69 y 70 estén cerrados. Las reservas gRPC tampoco implementan el canal de alertas solicitado por 53 y 73–75. Estos alcances distintos justifican no colapsar observaciones relacionadas como si fueran equivalentes.

La tabla 18 cubre los diez abiertos del registro `cierre/issues-corte.json` y los pendientes de Observaciones 2. Se separan esfuerzo de resolución y coste de arrastre: el primero estima trabajo para cerrar; el segundo describe el perjuicio mientras se posterga. No hay mediciones para monetizar ese perjuicio ni convertirlo en horas semanales. Los rangos son orientativos documentales, salvo las 8–12 h de UDP, declaradas en la respuesta al issue 78. No son horas ejecutadas ni compromisos aceptados por integrantes. Las áreas indicadas son responsables funcionales propuestos; la asignación personal corresponde al equipo.

Tabla 18: Deuda técnica: resolución, coste de arrastre y evidencia necesaria para cerrar.

Origen	Pendiente y esfuerzo de resolución	Coste de arrastre mientras se difiere	Área y condición de cierre
43	Clave idempotente entre clientes e inventario. 6–10 h.	Un reintento puede repetir descuentos; requiere conciliación y corrección de stock. La idempotencia del receptor aislado no protege la cadena.	Backend y pruebas: clave estable por operación y prueba de reintento sin doble movimiento.
53, 56, 73–75	Alertas de stock, suscriptor, deduplicación y reconexión. 20–32 h para el grupo.	Reposición manual y posible demora ante stock bajo; implementar solo el canal sin deduplicar expone a órdenes repetidas.	Backend: contrato de alertas, servidor y cliente que delegue en <code>OrdenCompraService</code> ; pruebas de reconexión y varios suscriptores.
64, 76	DTO de entrada y salida aún parciales. 8–12 h para el grupo.	Validación dispersa, contratos ambiguos y más retrabajo al modificar clientes o servicios.	Backend y pruebas: sustituir mapas/JSON crudo por DTO validados y probar entradas inválidas.
77	Rutas heredadas de productos. 3–5 h.	Documentación divergente y riesgo de romper consumidores al retirar rutas sin migración.	Backend y documentación: inventario de rutas, decisión de compatibilidad y OpenAPI concordante.
78	Telemetría UDP diferida por no ser prioritaria. 8–12 h, según respuesta publicada.	Se mantiene ausente ese canal; no se ha demostrado perjuicio operacional adicional con la instrumentación HTTP actual. Reactivarlo requiere revisar contrato, red, pérdida de datagramas y operación.	Arquitectura/operaciones: la decisión y su coste quedan documentados aquí. El cierre administrativo sigue pendiente; no se afirma implementación UDP.

Origen	Pendiente y esfuerzo de resolución	Coste de arrastre mientras se difiere	Área y condición de cierre
C3, C6	El oráculo retrospectivo detectó 13210 inconsistencias de inventario; pago independiente y compensación cancelada siguen no verificables.	Sin ledger de cobros ni estados durables de compensación no puede cerrarse el ciclo completo; sin partición controlada no se valida CAP.	Persistir identificadores e historiales de pago y compensación, añadir controles positivos e inducir una partición de red antes de afirmar garantías.
Eval. E2	Integración y validación automática de referencias; sin estimación acordada.	El CSV de seguridad y los tres enlaces web se verificaron por separado; ese control puntual no acredita todos los destinos del documento.	Integración: acordar el formato con la tarea 2, ejecutar el validador cuando esté disponible y regenerar el PDF final.

Las observaciones de evaluación E4 (GHCR), E6 (APK y checksum) y E8 (carpetas huérfanas) cuentan con los commits registrados en la tabla 2 y no constituyen pendientes documentales de esta tabla. Estos códigos de evaluación no designan entregas. Eval. E2 tiene resuelta la cita de seguridad: el CSV del 11 de septiembre registra ocho comprobaciones HTTP 401 y coincide con las ocho rutas del test parametrizado, dentro de una suite de once pruebas. La evidencia fue publicada en [b8d3e23](#) y sustituye la referencia histórica al CSV ausente del 1 de septiembre. El reporte local de la suite indica once pruebas sin fallos, pero no está versionado y no certifica por sí solo la fecha ni la revisión ejecutada. La corrección de esta cita no cierra todos los enlaces de E2 ni acredita un validador automático: los tres destinos web se corrigen y comprueban por separado en esta revisión; la integración y la validación automática permanecen pendientes. El cierre de E6 acredita un paquete release firmado, su integridad, procedencia y publicación automática. José confirmó su instalación e inicio en dos dispositivos; esa confirmación no acredita una prueba integral de compra ni registra modelos o versiones de Android.

Los costes de los grupos no se suman por cada issue: 53, 56 y 73–75 corresponden a una misma funcionalidad, al igual que 64 y 76. Los pendientes experimentales se relacionan con idempotencia y recuperación; se requiere planificación conjunta antes de calcular un total. No se modificaron los estados remotos de los issues durante esta revisión documental.

C2, C7, P3 y C9 se retiran de la tabla de deuda vigente porque ya cuentan con evidencia versionada: campaña con preflight, duración completa y rampa; cobertura verificable y CI rojo/verde; panel y trazas bajo carga; arranque desde volúmenes limpios; y cinco referencias específicas de coordinación. C3 y C6 permanecen por el alcance parcial del oráculo: ya existen hallazgos persistentes de facturación e inventario, pero faltan cobros, compensaciones durables y una partición controlada para evaluar CAP.

El issue 65 se rebate porque condicionaba el estado formal al seguimiento en tiempo real, que no se acredita como implementado. Esta justificación no elimina la necesidad de estados de negocio en el laboratorio; son alcances diferentes. Los costes técnicos reales deberán reestimarse al ejecutar cada cambio y sus pruebas.

12. Conclusiones

12.1. Respuestas a las preguntas de investigación

Pregunta 1: invariantes. La respuesta es parcial y revela fallos reales. El oráculo retrospectivo halló 13210 órdenes con movimiento de inventario ausente o distinto entre 43168 órdenes persistidas y 104 corridas con alguna violación observable. No halló discordancias de importe o líneas, descuentos duplicados ni stock negativo. No puede acreditar captura bancaria ni compensación completa porque esos estados no se persistieron. Por tanto, la implementación no conservó todas las invariantes y la evidencia restante se declara `no_verificado`.

Pregunta 2: rendimiento. La campaña permite una comparación descriptiva condicionada. Saga presenta menor mediana p95 en diez de doce condiciones, empate en una y valor mayor en una; además confirma 63,4 % de sus checkouts frente a 58,1 % de 2PC en el agregado. Ninguna prueba exacta por permutación resulta significativa con Bonferroni, por lo que no se afirma superioridad poblacional. La saturación reduce la confirmación de 94,1 % a 50 usuarios a 10,8 % a 400 para ambas estrategias agregadas.

Pregunta 3: recuperación. La campaña ejercitó omisión y temporización. El outbox persistente permitió medir convergencia factura-inventario en 37 de 60 corridas Saga, con una mediana global de 20602253 ms afectada por la acumulación del backlog. Ese valor no demuestra compensación de checkouts cancelados: sus estados vivían en memoria y se perdieron entre reinicios. No se atribuye recuperación completa a ninguna estrategia.

Pregunta 4: compatibilidad y RAG. El evaluador obtuvo 120/120 aciertos, pero ejecuta reglas locales del propio banco. La respuesta es negativa respecto de validar el asistente desplegado o comparar reglas con RAG. No se dispone de respuestas de ambos enfoques sobre el mismo conjunto independiente.

Pregunta 5: reproducibilidad distribuida. La campaña correctiva conserva 120 filas únicas, 24 condiciones con cinco repeticiones, 60 segundos de calentamiento, 300 segundos de medición y concurrencias efectivamente alcanzadas. El preflight, los pilotos, la rampa, el CSV crudo, la validación, los resúmenes y los scripts derivados están versionados en `experiments/paso8/resultados-reales/correctiva-20260905-final-v2/`. Esto permite auditar estructura y cálculos; los secretos, JWT y logs detallados se excluyen deliberadamente.

12.2. Balance acumulativo

TiendaTech pasó de la propuesta arquitectónica a una implementación con clientes, servicios, datos distribuidos e instrumentos de prueba. La campaña correctiva muestra que el flujo basal funciona y completa decenas de miles de compras, además de cuantificar su degradación con la concurrencia. El avance es verificable y el oráculo descubre una inconsistencia material de inventario; no elimina las incidencias abiertas ni permite convertir diferencias descriptivas sin significación corregida o evidencia parcial en garantías generales de una estrategia.

El análisis de Spark no mostró aceleración de T1 sin particionamiento JDBC al aumentar

ejecutores. El clúster conservó una consulta durante el fallo ensayado, con un pico de latencia. Ambas conclusiones son útiles aunque no sean una confirmación de las expectativas iniciales: permiten priorizar medición y coste frente a complejidad arquitectónica.

El Paso 13 queda atendido como consolidación documental con límites explícitos. No se declara que el proyecto cumpla todos los requisitos técnicos ni que se haya obtenido aprobación de similitud, autoría o evaluación docente. La continuidad requiere resolver la deuda registrada y sustituir cada resultado parcial por evidencia del mismo alcance.

El piloto posterior de la sección 7.4 aporta una respuesta acotada adicional a la pregunta de invariantes: el nuevo control detecta una discordancia entre inventario y movimientos en E-Saga local. Este hallazgo impide extender el cero histórico al banco corregido, pero no sustituye la campaña de comparación ni valida garantías distribuidas. Las correcciones de código y los resultados de esa futura campaña deben evaluarse por separado.

13. Conclusiones individuales

Las siguientes reflexiones conservan y actualizan los textos individuales de la memoria anterior según las evidencias del corte. José y Andy comunicaron la revisión de sus textos y declaraciones de IA; Jhinson y Jeremy mantienen la responsabilidad de revisar los suyos antes de la entrega institucional. Esta constancia no sustituye la conformidad o firma que exija el SGA.

13.1. Jhinson Stalyn Aucatoma Celorio

La evolución de TiendaTech me permitió comprender que una aplicación distribuida no se considera terminada únicamente porque sus servicios respondan y sus contenedores puedan levantarse. La Entrega 4 hizo visible la importancia de convertir decisiones técnicas en controles repetibles: una regla de negocio debe tener pruebas, una modificación debe atravesar un pipeline y un problema operativo debe poder observarse mediante datos. El refactor por capas fue especialmente valioso porque separó responsabilidades que antes estaban mezcladas y permitió probar la lógica sin depender de la base de datos o de otros servicios. También aprendí que la cobertura es útil como señal, pero no sustituye el diseño de casos relevantes; alcanzar el setenta por ciento tiene sentido solamente si se cubren decisiones que afectan inventario, órdenes, facturación y usuarios. La preparación de Prometheus, logs estructurados y Grafana mostró que medir un sistema requiere definir antes qué pregunta se desea responder. Finalmente, la evaluación ISO/IEC 25010 reforzó la necesidad de informar resultados honestos: una métrica pendiente debe conservarse como pendiente y no completarse con una estimación. Considero que el proyecto logró una base más mantenible y verificable, aunque todavía necesita ejecutar la observación prolongada, la carga oficial del despliegue, aunque las últimas ejecuciones de CI ya fueron verificadas. Mi principal aprendizaje es que la calidad no es una actividad final, sino una propiedad que debe incorporarse al flujo normal de desarrollo. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

13.2. Jeremy Ruperto Gaibor Rodriguez

Durante este proyecto confirmé que el trabajo con microservicios exige coordinar elementos que en una aplicación monolítica suelen permanecer juntos. El API Gateway, los contratos HTTP, la autenticación y la persistencia distribuida forman una cadena: un error en cualquiera de esos puntos puede afectar un flujo completo aunque cada servicio funcione por separado. Las pruebas de integración añadidas ayudan a reducir esa incertidumbre porque verifican recorridos reales hacia productos, usuarios y pedidos, mientras que las pruebas de seguridad comprueban que las rutas protegidas no queden expuestas accidentalmente. El pipeline representa otro cambio importante en la forma de trabajar. Ya no basta con que una modificación compile en la computadora de un integrante; las mismas pruebas, reglas de cobertura y construcciones deben ejecutarse en un ambiente reproducible. También comprendí el valor de hacer depender el build de los controles anteriores, pues publicar una imagen cuando existen pruebas fallidas trasladaría el riesgo hacia producción. La evaluación ISO reveló además que aprobar cobertura no implica aprobar mantenibilidad: el diagnóstico inicial de PMD encontró complejidad de 20, mientras que el refactor posterior dejó los máximos medidos entre siete y nueve en Java y en nueve en la web. Por ello, mi conclusión es que el proyecto avanzó de una demostración funcional hacia un producto con mecanismos de control, aunque su validación final depende de conservar evidencias reales, completar las mediciones operativas pendientes y mantener bajo control la complejidad después del refactor. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

13.3. Andy Paul Sanchez Pilaloa

El desarrollo de TiendaTech me ayudó a relacionar conceptos de arquitectura, pruebas y operación que anteriormente podía considerar independientes. La arquitectura en capas facilitó localizar la lógica de negocio y diseñar pruebas unitarias con dependencias simuladas. Esa separación también disminuye el costo de reemplazar una implementación de persistencia o comunicación, porque el dominio no necesita conocer detalles de infraestructura. La parte de observabilidad resultó igualmente significativa: un contador permite conocer la demanda, un histograma permite analizar percentiles y un gauge representa valores que suben y bajan, como las conexiones activas. Estas diferencias son esenciales para construir un dashboard que sirva para tomar decisiones y no sea solamente decorativo. La instrumentación desarrollada deja logs JSON y métricas homogéneas en siete servicios, además de Prometheus y Alloy como componentes de recolección. Sin embargo, configurar herramientas no equivale a completar la observabilidad: la correlación por trazas y la captura de Grafana con datos reales, que en un primer momento quedaron pendientes, ya se cerraron con evidencia real; la disponibilidad de una hora ya se midió después, con resultado favorable (100 % sobre 3600 s continuos); permanece pendiente el p95 productivo bajo múltiples orígenes de carga. El modelo ISO/IEC 25010 y los umbrales académicos aportaron criterios para expresar esa diferencia mediante evidencia. El aprendizaje más importante fue reconocer que documentar una limitación también es una práctica profesional. Un reporte técnico útil distingue con claridad lo implementado, lo ejecutado y lo todavía pendiente, permitiendo que otra persona continúe el trabajo sin depender de

afirmaciones imposibles de reproducir. Además, participar en las pruebas de seguridad, cobertura y carga me enseñó que cada resultado necesita un contexto reproducible, criterios de aceptación definidos y seguimiento de fallos antes de convertirse en una afirmación de calidad. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

13.4. José Alejandro Lozano Morales

La construcción acumulativa del PFC me permitió comprobar que la documentación pierde valor cuando permanece fija mientras el sistema y su evidencia cambian. Como responsable documental tuve que conciliar entregas, commits, incidencias, tablas, figuras y resultados sin convertir una expectativa técnica en un hecho medido. La campaña correctiva hizo visible esa responsabilidad: primero sustituyó un ensayo con apenas tres confirmaciones por 120 corridas distribuidas y 30275 checkouts confirmados; después, el oráculo retrospectivo sobre CockroachDB reveló 13210 órdenes con un movimiento de inventario ausente o distinto. El hallazgo obliga a informar una inconsistencia real aunque perjudique una conclusión favorable sobre 2PC o Saga.

También aprendí que un resultado cero necesita límites. La instantánea no mostró importes de factura discordantes, descuentos duplicados ni stock negativo, pero el sistema no conservó un ledger de cobros ni los estados durables necesarios para demostrar la compensación de intentos cancelados. Del mismo modo, observar invariantes bajo carga no valida una posición CAP sin inducir una partición de red. Mi aporte consistió en mantener esas fronteras visibles, actualizar la trazabilidad y la deuda técnica y conservar cálculos reproducibles sin alterar los datos crudos. La revisión de credenciales, tokens, capturas y declaraciones de IA reforzó que la transparencia también protege a usuarios y autores. Concluyo que documentar profesionalmente exige revisar la evidencia hasta el último corte, corregir afirmaciones anteriores y comunicar con la misma claridad los logros, los fallos encontrados y aquello que continúa como `no_verificado`.

14. Reflexión ética

El Código de Ética y Práctica Profesional de Ingeniería de Software de ACM/IEEE-CS [23] sitúa el interés público, la calidad del producto y el juicio profesional como responsabilidades del desarrollo. En TiendaTech se traducen en decisiones concretas: no presentar pagos simulados como integración bancaria, no convertir un porcentaje de cobertura acotado en garantía de todo el sistema y no ocultar fallos para obtener una evaluación más favorable.

Las reservas y pagos tienen consecuencias potenciales para usuarios. Un reintento sin deduplicación puede descontar existencias de nuevo; una compensación incompleta puede mostrar una compra fallida con efectos persistentes. Aunque el proyecto sea académico, su documentación debe distinguir esos riesgos y explicar por qué no se recomienda usar un ensayo local como certificación productiva. Informar de un resultado negativo es una contribución técnica cuando impide una decisión basada en confianza falsa.

El manejo de credenciales y datos personales exige mínima exposición. Los secretos se suministran por configuración y no deben aparecer en commits, capturas ni logs. Antes de compartir evidencias se revisan tokens, direcciones y otros identificadores. Los datos sintéticos se rotulan como tales para evitar que se interpreten como transacciones de personas o estadísticas de un negocio real. La replicación no es una excusa para conservar innecesariamente información sensible.

La honestidad bibliográfica requiere comprobar que cada identificador corresponde a la obra citada. Reutilizar una referencia sin verificarla puede atribuir a un autor resultados que nunca publicó. Esta revisión corrigió la selección de fuentes académicas y mantuvo los estándares mediante enlaces institucionales, sin inventar DOI para cumplir una casilla formal.

El uso de IA generativa no transfiere la autoría ni la responsabilidad al asistente. Las sugerencias deben contrastarse con código y resultados; una redacción convincente puede ocultar supuestos incorrectos. Tampoco es admisible atribuir experiencias personales, firmas o aceptación a otros integrantes sin su revisión. Las reflexiones individuales se presentan para revisión y no sustituyen ese consentimiento.

Finalmente, la urgencia de entrega no justifica cambiar datos ni cerrar incidencias sin evidencia. Este informe conserva trabajo pendiente, acota estimaciones y no declara un porcentaje de similitud que no se ha medido. La obligación profesional es comunicar lo conocido y lo desconocido con suficiente claridad para que otra persona pueda evaluar el producto y continuar sin repetir errores.

15. Declaraciones

15.1. Autoría y contribución

El equipo AGLS está compuesto por los cuatro integrantes de la portada, con roles asignados desde la primera entrega. La tabla 19 resume responsabilidades y evidencia de trabajo colectivo, sin asignar porcentajes ni afirmar exclusividad sobre archivos desarrollados conjuntamente.

Tabla 19: Contribuciones por rol, sujetas a revisión final de sus autores.

Integrante	Contribución documentada
Jhinson Aucatoma	Arquitectura, ADR, modelo y distribución de datos; organización de evidencias de clúster y decisiones.
Jeremy Gaibor	Desarrollo e integración de servicios, comunicaciones y banco experimental; scripts y resultados incorporados al repositorio.
Andy Sanchez	Pruebas, cobertura, complejidad, CI, seguridad, carga e instrumentación; informes con alcance y límites.
José Lozano	Denominación y trazabilidad, revisión documental y de incidencias, consolidación bibliográfica y cierre acumulativo.

La contribución de una persona no excluye ayuda de otra. Los commits y las respuestas

de revisión constituyen evidencia de evolución, no una medición automática de esfuerzo individual. Esta memoria no añade firmas digitales ni certifica una aprobación que no haya sido comunicada.

15.2. Uso de inteligencia artificial generativa

Las herramientas declaradas por el documentalista para el equipo son Claude para Jhinson y Andy, y Codex para Jeremy, Andy y José. No se incorporan otras herramientas generativas por suposición. Se usaron como apoyo para análisis técnico, desarrollo, revisión y redacción, no como sustitución del trabajo ni de la responsabilidad humana. Jhinson empleó Claude en materiales de arquitectura y decisiones, cuya coherencia con los ADR y el despliegue revisó personalmente; Jeremy empleó Codex en implementación y banco de pruebas, y revisó su ejecución, código y resultados; Andy empleó Claude y Codex como apoyo en calidad, CI, seguridad, carga y revisión de su conclusión individual, y contrastó las afirmaciones con pruebas e informes; José empleó Codex en trazabilidad, revisión documental, bibliografía, tablas, figuras y LaTeX, y comprobó fuentes, rutas, compilación y presentación del PDF.

En el cierre documental, Codex asistió a José en la inspección de evidencias, conciliación de resultados, preparación de tablas y figuras, revisión bibliográfica, edición de LaTeX y comprobación del PDF; también asistió a Andy en la revisión y ampliación de su conclusión individual y en la actualización de su declaración de IA. Las secciones afectadas abarcan arquitectura, diseño del estudio, resultados, calidad, gestión de la revisión, conclusiones y declaraciones. No se atribuye un modelo, versión, prompt o procedimiento de validación más específico que el comunicado por cada integrante.

José y Andy revisaron sus textos individuales y declaraciones de IA; Jhinson y Jeremy deben completar esa lectura antes de presentarlos como declaraciones personales. La generación asistida no demuestra ausencia de errores: las afirmaciones se mantienen vinculadas a archivos, y los resultados faltantes se declaran como no medidos. La comprobación institucional de similitud inferior al 15 % no se ejecutó en este cierre y no se certifica.

15.3. Reproducibilidad documental

La fuente principal es `docs/entrega4/PFC4.tex` y utiliza la bibliografía compartida `docs/entrega3/referenciasPFC.bib`. La compilación se realiza desde `docs/entrega4/` mediante pdfLaTeX, Biber y dos pasadas adicionales de pdfLaTeX, según las instrucciones del README principal. Las imágenes se resuelven con rutas relativas y los diagramas C4 se generan desde TikZ.

Para reproducir el PDF desde un clon se versionan el logo, las imágenes de `img/` y `release/screenshots/`, y las figuras derivadas. La evidencia distribuida principal conserva las 120 corridas de microservicios y CockroachDB, preflight, pilotos, rampa, resumen y validación en `experiments/paso8/resultados-reales/correctiva-20260905-final-v2/.analyze_corrective_comparison.py`, `plot_corrective_results.py`, `reconstruct_real_oracle.py` y `analyze_corrective_results.py` regeneran los resúmenes, el oráculo retrospectivo y las figuras sin modificar el CSV crudo. El banco SQLite permanece como piloto histórico y no sustenta C2, C3 ni C6.

El README de esta entrega fija CPython 3.11.1, Matplotlib 3.9.0 y el identificador de

la imagen TeX Live 2026; el cuaderno puede verificarse con el ejecutor estándar incluido, sin instalar Jupyter. También proporciona desde la clonación los comandos de prueba, regeneración, análisis y compilación. El 1 de septiembre de 2026, una clonación local aislada del árbol candidato completó tres pruebas, 120 corridas, el cuaderno, las figuras y la compilación de 39 páginas en 623,55 segundos (10 min 23,55 s), con 120/120 aprobaciones del oráculo y cero inconsistencias observadas. La descarga dependiente de la red se excluyó del cronómetro y el resultado no se equipara a una ejecución del sistema distribuido productivo.

El hash enlazado identifica las fuentes remotas de las mediciones. El cierre se integra en los nombres originales del documento y la bibliografía; las versiones anteriores permanecen consultables en el historial de Git.

16. Bibliografía

- [1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*. Springer International Publishing, 2020. DOI: [10.1007/978-3-030-26253-2](https://doi.org/10.1007/978-3-030-26253-2)
- [2] R. Taft et al., «CockroachDB: The Resilient Geo-Distributed SQL Database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, págs. 1493-1509. DOI: [10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134)
- [3] S. Gilbert y N. Lynch, «Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601)
- [4] E. Brewer, «CAP twelve years later: How the “rules” have changed,» *Computer*, vol. 45, n.º 2, págs. 23-29, 2012. DOI: [10.1109/MC.2012.37](https://doi.org/10.1109/MC.2012.37)
- [5] D. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: [10.1109/MC.2012.33](https://doi.org/10.1109/MC.2012.33)
- [6] L. Lamport, «Time, clocks, and the ordering of events in a distributed system,» *Communications of the ACM*, vol. 21, n.º 7, págs. 558-565, 1978. DOI: [10.1145/359545.359563](https://doi.org/10.1145/359545.359563)
- [7] H. Garcia-Molina y K. Salem, «Sagas,» en *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, ACM, 1987, págs. 249-259. DOI: [10.1145/38713.38742](https://doi.org/10.1145/38713.38742)
- [8] J. Gray y L. Lamport, «Consensus on Transaction Commit,» *ACM Transactions on Database Systems*, vol. 31, n.º 1, págs. 133-160, 2006. DOI: [10.1145/1132863.1132867](https://doi.org/10.1145/1132863.1132867)
- [9] M. Štefanko, O. Chaloupka y B. Rossi, «The Saga Pattern in a Reactive Microservices Environment,» en *Proceedings of the 14th International Conference on Software Technologies*, SCITEPRESS, 2019, págs. 483-490. DOI: [10.5220/0007918704830490](https://doi.org/10.5220/0007918704830490)
- [10] K. Dürr, R. Lichtenthäler y G. Wirtz, «Saga Pattern Technologies: A Criteria-based Evaluation,» en *Proceedings of the 12th International Conference on Cloud Computing and Services Science*, SCITEPRESS, 2022, págs. 141-148. DOI: [10.5220/0010999400003200](https://doi.org/10.5220/0010999400003200)
- [11] E. Daraghmi, C.-P. Zhang y S.-M. Yuan, «Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture,» *Applied Sciences*, vol. 12, n.º 12, 2022, núm. 6242. DOI: [10.3390/app12126242](https://doi.org/10.3390/app12126242)
- [12] G. M. Amdahl, «Validity of the single processor approach to achieving large scale computing capabilities,» en *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*, ACM Press, 1967, pág. 483. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560)
- [13] J. L. Gustafson, «Reevaluating Amdahl’s law,» *Communications of the ACM*, vol. 31, n.º 5, págs. 532-533, 1988. DOI: [10.1145/42411.42415](https://doi.org/10.1145/42411.42415)
- [14] M. Rigger y Z. Su, «Finding bugs in database systems via query partitioning,» *Proceedings of the ACM on Programming Languages*, vol. 4, n.º OOPSLA, págs. 1-30, 2020. DOI: [10.1145/3428279](https://doi.org/10.1145/3428279)

- [15] G. Marquez, F. Osses y H. Astudillo, «Review of Architectural Patterns and Tactics for Microservices in Academic and Industrial Literature,» *IEEE Latin America Transactions*, vol. 16, n.º 9, págs. 2321-2327, 2018. DOI: [10.1109/TLA.2018.8789551](https://doi.org/10.1109/TLA.2018.8789551)
- [16] C. Zhong et al., «Domain-Driven Design for Microservices: An Evidence-Based Investigation,» *IEEE Transactions on Software Engineering*, vol. 50, n.º 6, págs. 1425-1449, 2024. DOI: [10.1109/TSE.2024.3385835](https://doi.org/10.1109/TSE.2024.3385835)
- [17] P. Nawrocki, K. Wrona, M. Marczak y B. Sniezynski, «A Comparison of Native and Cross-Platform Frameworks for Mobile Applications,» *Computer*, vol. 54, n.º 3, págs. 18-27, 2021. DOI: [10.1109/MC.2020.2983893](https://doi.org/10.1109/MC.2020.2983893)
- [18] M. Shahin, M. Ali Babar y L. Zhu, «Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices,» *IEEE Access*, vol. 5, págs. 3909-3943, 2017. DOI: [10.1109/ACCESS.2017.2685629](https://doi.org/10.1109/ACCESS.2017.2685629)
- [19] P. Rostami Mazrae, T. Mens, M. Golzadeh y A. Decan, «On the usage, co-usage and migration of CI/CD tools: A qualitative analysis,» *Empirical Software Engineering*, vol. 28, n.º 2, 2023. DOI: [10.1007/s10664-022-10285-5](https://doi.org/10.1007/s10664-022-10285-5)
- [20] M. Hui et al., «Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,» *Journal of Systems and Software*, vol. 220, pág. 112 232, 2025. DOI: [10.1016/j.jss.2024.112232](https://doi.org/10.1016/j.jss.2024.112232)
- [21] ISO/IEC. «ISO/IEC 25010:2023: Product quality model,» visitado 1 de sep. de 2026. dirección: <https://www.iso.org/standard/78176.html>
- [22] ISO/IEC. «ISO/IEC 25023:2016: Measurement of system and software product quality,» visitado 1 de sep. de 2026. dirección: <https://www.iso.org/standard/35747.html>
- [23] ACM/IEEE-CS. «Software Engineering Code of Ethics and Professional Practice, Version 5.2,» visitado 1 de sep. de 2026. dirección: <https://www.acm.org/code-of-ethics/software-engineering-code>